Grupo 7
Tomas Diaz – 202520393
Juan Felipe Moreno – 202521572
David Ortiz – 202520816

1. Introducción
1.1. Propósito del documento

2. Base de análisis corregida
2.1. Cambios realizados a partir de la Entrega 1

3. Requerimientos funcionales
3.1. Requerimientos del Socio
3.2. Requerimientos de los Empleados
3.3. Requerimientos del Administrador

4. Reglas de dominio
4.1. Equipos y jugadores
4.2. Competencias
4.3. Deportes individuales
4.4. Entrenamientos
4.5. Tienda y cafetería
4.6. Administración y turnos
4.7. Usuarios y autenticación

5. Restricciones del sistema
5.1. Restricciones técnicas
5.2. Restricciones de alcance
5.3. Persistencia de información

6. Diseño general de la solución
6.1. Organización del sistema
6.2. Componentes principales
6.3. Distribución de responsabilidades

7. Diagrama UML de diseño de alto nivel
7.1. Vista general
7.2. Usuarios y gestión de turnos
7.3. Deportes de conjunto y competencias
7.4 Disciplinas individuales
7.5 Tienda, cafetería e inventario
7.6 Persistencia y programas de prueba

8. Diagrama UML detallado
8.1. Vista general
8.2. Usuarios y gestión de turnos
8.3. Deportes de conjunto y competencias
8.4 Disciplinas individuales
8.5 Tienda, cafetería e inventario
8.6 Persistencia y programas de prueba

9. Asignación de responsabilidades
9.1. Responsabilidades por clase
9.2. Principios usados para distribuir responsabilidades

10. Justificación de decisiones de diseño
10.1. Separación de responsabilidades
10.2. Herencia y clases abstractas
10.3. Clases intermedias y relaciones
10.4. Distribución de las reglas de dominio
10.5. Organización de los diagramas e implementación

11. Diseño de persistencia
11.1. Información que debe almacenarse
11.2. Archivos utilizados
11.3. Carga de información
11.4. Guardado de información

12. Diagramas de secuencia
12.1. Requerimiento crítico 1
12.2. Requerimiento crítico 2
12.3. Requerimiento crítico 3

13. Instrucciones de ejecución

14. Conclusiones

15. Uso de Inteligencia Artificial Generativa

1. INTRODUCCIÓN

1.1 Propósito del documento

El propósito del documento es presentar de forma ordenada el diseño de la aplicación que permitirá gestionar la información del Club Deportivo Séneca; para esto, se parte del modelo de dominio que fue corregido y se avanza hacia una estructura que pueda convertirse posteriormente en clases, atributos, relaciones y métodos dentro del proyecto desarrollado en Java; también el documento busca dejar claro cómo cada parte del sistema responde a una necesidad del caso, por esto mismo, además de presentar los diagramas realizados, se explican las decisiones que tomamos y se relacionan con los requerimientos funcionales diseñados previamente, las reglas de dominio y las restricciones técnicas.

2. BASE DE ANÁLISIS CORREGIDA

2.1. Cambios realizados a partir de la Entrega 1

A partir de la retroalimentación recibida y de una nueva revisión del caso, el grupo realizó los siguientes ajustes principales:

1. **Solicitudes de cambio de turno:** se añadió una entidad para representar las solicitudes realizadas por entrenadores y fisioterapeutas, así como su revisión por parte del administrador.

2. **Disciplinas individuales:** se separó la disciplina de la instalación y se creó una inscripción por disciplina, permitiendo manejar de forma independiente el nivel y el ranking de cada socio en tenis, pádel y tenis de mesa.

3. **Estadísticas deportivas:** se dejó de utilizar una única entidad con estadísticas mezcladas; en su lugar, se organizaron según el deporte, ya que fútbol, baloncesto, voleibol y las disciplinas individuales registran datos diferentes.

4. **Partidos y competencias:** se revisaron las relaciones entre Partido, PartidoInterno y PartidoCompetencia; adicionalmente, se incorporó la inscripción del equipo y la nómina de jugadores dentro de cada competencia, ya que únicamente los jugadores registrados pueden disputar partidos oficiales.

5. **Ventas:** se añadió DetalleVenta para registrar la cantidad y el precio de cada producto vendido, evitando manejar directamente una relación muchos a muchos entre venta y producto.

6. **Inventario:** se amplió el modelo para permitir manejar existencias por ubicación, debido a que el administrador puede mover artículos entre diferentes lugares del inventario.

7. **Información necesaria para la implementación:** se incorporaron datos como fecha de nacimiento del socio, límites de edad por categoría, estado activo de jugadores y equipos, sugerencias de productos, mensualidades y acumulación de sanciones; de esta manera, el modelo cuenta con la información necesaria para aplicar posteriormente las reglas del sistema.

3. REQUERIMIENTOS FUNCIONALES

3.1. Requerimientos funcionales del socio

RF-SOC-01. El sistema debe permitir al socio inscribirse en deportes de conjunto y pertenecer a un equipo por cada deporte, verificando que su edad corresponda a la categoría correspondiente.

RF-SOC-02. El sistema debe permitir al socio participar en partidos internos y consultar el registro de sus estadísticas y resultados.

RF-SOC-03. El sistema debe permitir al socio inscribirse en las academias de tenis, pádel y tenis de mesa, asignándole un nivel correspondiente a Principiante, Intermedio o Avanzado.

RF-SOC-04. El sistema debe permitir al socio reservar una instalación para una disciplina individual, verificando disponibilidad, capacidad, horario y modalidad.

RF-SOC-05. El sistema debe permitir al socio programar partidos de práctica con otros jugadores, validando la compatibilidad entre niveles y la excepción aplicable cuando uno de los participantes sea entrenador.

RF-SOC-06. El sistema debe permitir registrar las estadísticas de los partidos de práctica y actualizar el ranking correspondiente a cada jugador y disciplina.

RF-SOC-07. El sistema debe permitir al socio comprar artículos de la tienda, verificando que exista inventario suficiente y aplicando los impuestos y descuentos correspondientes.

RF-SOC-08. El sistema debe permitir al socio realizar compras en la cafetería; antes de confirmar una compra de snacks, deberá mostrar la información relacionada con sus posibles alérgenos.

RF-SOC-09. El sistema debe aplicar las restricciones de seguridad relacionadas con bebidas calientes en las instalaciones donde no están permitidas.

RF-SOC-10. El sistema debe permitir al socio utilizar los puntos de fidelidad acumulados como descuento en compras posteriores.

3.2 Requerimientos funcionales de los empleados

RF-EMP-01. El sistema debe permitir al entrenador programar sesiones de entrenamiento, indicando fecha, hora, duración, instalación y jugadores convocados.

RF-EMP-02. El sistema debe permitir al entrenador convocar a un equipo completo o a un subconjunto de jugadores para una sesión de entrenamiento.

RF-EMP-03. El sistema debe permitir al entrenador registrar la asistencia de los jugadores convocados como asistió, faltó o llegó tarde.

RF-EMP-04. El sistema debe permitir al entrenador registrar de manera opcional una evaluación de desempeño, utilizando una nota entre 1 y 5 y un comentario.

RF-EMP-05. El sistema debe permitir al entrenador verificar que un equipo tenga la cantidad mínima de jugadores habilitados antes de disputar un partido.

RF-EMP-06. El sistema debe permitir al entrenador registrar y consultar las estadísticas deportivas de los jugadores bajo su seguimiento.

RF-EMP-07. El sistema debe permitir a entrenadores y fisioterapeutas consultar sus turnos semanales y solicitar cambios sobre los turnos que tengan asignados.

RF-EMP-08. El sistema debe registrar las solicitudes de cambio de turno realizadas por entrenadores y fisioterapeutas, para que posteriormente puedan ser evaluadas por el administrador.

RF-EMP-09. El sistema debe permitir a los empleados realizar compras y aplicar el descuento del 15 % cuando utilicen su propio código.

RF-EMP-10. El sistema debe permitir que un socio utilice el código de descuento compartido por un empleado; en este caso se deberá aplicar un descuento del 8 %, siempre verificando que no sea acumulable con otras promociones.

RF-EMP-11. El sistema debe permitir a los empleados registrar sugerencias de nuevos artículos deportivos o snacks, las cuales posteriormente podrán ser aprobadas o rechazadas por el administrador.

3.3 Requerimientos funcionales del administrador

RF-ADM-01. El sistema debe permitir al administrador crear y dar de baja jugadores y equipos.

RF-ADM-02. El sistema debe permitir al administrador gestionar la inscripción de equipos y jugadores en competencias externas, verificando el cumplimiento de sus requisitos.

RF-ADM-03. El sistema debe permitir al administrador consultar y gestionar el inventario de artículos e instalaciones.

RF-ADM-04. El sistema debe permitir al administrador mover artículos entre diferentes ubicaciones del inventario y reabastecer la tienda.

RF-ADM-05. El sistema debe permitir al administrador aprobar o rechazar las sugerencias de artículos deportivos o snacks realizadas por los empleados.

RF-ADM-06. El sistema debe permitir al administrador gestionar los turnos de entrenadores y fisioterapeutas.

RF-ADM-07. El sistema debe validar que cualquier cambio de turno aprobado mantenga disponibles como mínimo dos entrenadores y un fisioterapeuta durante el horario de atención.

RF-ADM-08. El sistema debe permitir al administrador consultar el historial de reservas, partidos y sesiones de entrenamiento.

RF-ADM-09. El sistema debe permitir al administrador consultar reportes financieros discriminados por rubro, incluyendo tienda, cafetería y mensualidades; además, estos reportes podrán consultarse por periodo diario, semanal o mensual.

RF-ADM-10. El sistema debe permitir al administrador consultar reportes deportivos relacionados con asistencia a entrenamientos, resultados de partidos y evolución de rankings.

4. REGLAS DE DOMINIO

4.1 Equipos y jugadores

- El club ofrece fútbol, baloncesto y voleibol como deportes de equipo; los equipos se organizan en las categorías Sub-13, Sub-15, Sub-17 y Mayores, y cada equipo pertenece únicamente a un deporte y a una categoría.
- Cada equipo cuenta con un número máximo de jugadores; asimismo, un socio solamente puede pertenecer a un equipo por cada deporte, aunque puede participar simultáneamente en equipos de deportes distintos.
- La edad del jugador debe corresponder con la categoría del equipo; además, el número de camiseta debe ser único dentro del mismo equipo, de manera que dos jugadores pertenecientes al mismo equipo no pueden utilizar el mismo número.
- Para disputar un partido interno, ambos equipos deben pertenecer al mismo deporte y categoría; adicionalmente, deben cumplir con el mínimo de jugadores habilitados, correspondiente a siete jugadores para fútbol, cinco para baloncesto y seis para voleibol.
- Las estadísticas registradas dependerán del deporte; en fútbol se manejan goles, asistencias y tarjetas, en baloncesto puntos y rebotes, mientras que en voleibol se registran aces y bloqueos.
- En fútbol, un jugador que acumule dos tarjetas amarillas o reciba una tarjeta roja quedará suspendido para el siguiente partido correspondiente.

4.2 Equipos y jugadores

- Cada competencia debe contar con un nombre, entidad organizadora, año o temporada, deporte, categoría habilitada y formato de juego; a su vez, un equipo solamente puede ser inscrito cuando cumpla los requisitos definidos para esa competencia.
- La nómina debe contar con la cantidad mínima de jugadores habilitados; además, únicamente los jugadores incluidos en dicha nómina podrán participar en los partidos oficiales.
- Los partidos de competencia deben registrar fecha, marcador, institución rival y escenario; asimismo, el sistema deberá diferenciar el formato utilizado, ya que el Torneo de los Cerros funciona normalmente como una liga de todos contra todos, mientras que ASCUN puede manejar fase de grupos y posteriormente eliminación directa.
- En fútbol, una victoria otorga tres puntos, un empate concede un punto y una derrota no suma puntos; por otro lado, baloncesto y voleibol no permiten empates, por lo cual siempre debe existir un ganador. En el caso del voleibol, el marcador se lleva por sets.
- Las estadísticas y sanciones deben mantenerse durante toda la competencia; por esta razón, si un jugador se encuentra suspendido por acumulación de tarjetas, no podrá ser alineado en el siguiente partido oficial de su equipo.

4.3 Deportes individuales

- Las disciplinas individuales disponibles son tenis, pádel y tenis de mesa; cada socio inscrito debe contar con un nivel correspondiente a Principiante, Intermedio o Avanzado.

- Las instalaciones deben respetar su capacidad; una cancha de tenis permite dos o cuatro jugadores dependiendo de si se juega sencillos o dobles, una cancha de pádel admite cuatro jugadores, mientras que una mesa de tenis de mesa permite dos o cuatro participantes.
- Las reservas deben realizarse por bloques horarios; además, deben respetar la capacidad, la modalidad y el número máximo de instalaciones que pueden utilizarse simultáneamente para cada disciplina.
- Los partidos de práctica solamente podrán programarse entre jugadores del mismo nivel o de niveles adyacentes; sin embargo, esta restricción no aplica cuando uno de los participantes también sea entrenador.
- Después de cada partido, el sistema deberá actualizar el ranking correspondiente al jugador dentro de esa disciplina específica.

4.4 Entrenamientos

- Cada sesión de entrenamiento debe registrar fecha, hora, duración, entrenador responsable, instalación utilizada y jugadores convocados.
- En deportes de conjunto, el entrenador puede convocar al equipo completo o solamente a un subconjunto de sus jugadores; en las disciplinas individuales también pueden realizarse sesiones individuales o grupos pequeños.
- La asistencia solamente podrá registrarse para jugadores que hayan sido convocados; sus estados posibles serán asistió, faltó o llegó tarde.
- La evaluación de desempeño será opcional; cuando se registre, deberá utilizar una nota entre 1 y 5, pudiendo incluir también un comentario.

4.5 Tienda y cafetería

- Cada artículo de la tienda debe contar con nombre, categoría, precio y disponibilidad de inventario; una compra solamente podrá realizarse cuando exista cantidad suficiente del producto solicitado.
- Las compras realizadas en la tienda aplican un IVA del 19 %; además, los empleados reciben un descuento del 15 % en sus propias compras y pueden compartir un código que otorga un descuento del 8 % a otros socios. Estos descuentos no podrán acumularse con otras promociones.
- Los snacks deben mantener registrada la información relacionada con sus posibles alérgenos; esta información deberá mostrarse al socio antes de confirmar la compra.
- Las bebidas pueden ser frías o calientes; sin embargo, no se permite despachar una bebida caliente a un socio que se encuentre dentro de una cancha de pádel o de una mesa de tenis de mesa techada, de la misma manera, tampoco se permitirá utilizar dichas instalaciones mientras el socio tenga consigo una bebida caliente.
- Las ventas de cafetería aplican un impuesto al consumo del 8 %; adicionalmente, existe una propina sugerida del 10 %, aunque el socio puede modificar este valor.
- Finalmente, cada venta debe registrar sus diferentes rubros y generar puntos de fidelidad equivalentes al 2 % de su valor; un punto equivale a un peso y podrá ser utilizado posteriormente como descuento.

4.6 Administración y turnos

- Durante todo el horario de atención deben permanecer disponibles como mínimo dos entrenadores y un fisioterapeuta; por esta razón, una solicitud de cambio de turno solamente podrá ser aprobada cuando no rompa esta condición.

- El administrador podrá gestionar jugadores, equipos, inventarios y turnos; además, tendrá acceso a los reportes administrativos y deportivos establecidos.
- Sin embargo, el administrador no podrá inscribirse en partidos ni reservar instalaciones para uso personal, manteniendo así separadas sus funciones administrativas de las actividades propias de los socios.

4.7 Usuarios, autenticación y persistencia

- Todos los usuarios del sistema deben contar con un login y un password; por esto mismo, la aplicación deberá conservar la información necesaria para identificar a cada usuario y reconocer su tipo dentro del sistema.
- Asimismo, la información debe mantenerse entre distintas ejecuciones de la aplicación; en consecuencia, los datos relevantes deberán guardarse y recuperarse utilizando archivos, evitando que el estado del sistema se pierda al cerrar el programa.
- Los archivos utilizados para este proceso deberán encontrarse en una carpeta diferente a la carpeta que contiene el código fuente de la aplicación.

5. RESTRICCIONES DEL SISTEMA

5.1 Restricciones técnicas

- La aplicación debe desarrollarse utilizando Java; de esta manera, todas las clases, relaciones y funcionalidades presentadas deberán ser usadas por este lenguaje.

- Toda la información relevante del sistema deberá ser persistente; se utilizarán archivos como almacenamiento, pudiendo existir varios archivos dependiendo de la forma en la que se organice la información.

- La carpeta utilizada para almacenar estos archivos deberá estar separada de la carpeta donde se encuentre el código fuente; adicionalmente, puede asumirse que los archivos solamente serán leídos o modificados por la propia aplicación, por lo cual no es necesario contemplar usuarios externos maliciosos.

- Todos los usuarios deberán contar con login y password; la solución deberá conservar la información necesaria para identificar socios, empleados y administrador.

5.2 Restricciones de alcance

- Para esta etapa del proyecto no se requiere una interfaz gráfica; en su lugar, el funcionamiento de la aplicación será demostrado mediante programas ejecutados desde consola. (Primera entrega)

- Estos programas deberán solicitar únicamente la información necesaria para realizar cada demostración; asimismo, deberán mostrar de manera clara el estado de la aplicación y permitir comprobar que las funcionalidades están trabajando correctamente.

- Aunque todavía no se implemente una interfaz completa para los usuarios, la lógica necesaria para soportar las funcionalidades deberá quedar desarrollada; esto permitirá que en proyectos posteriores se pueda agregar una capa de interacción sin tener que reconstruir la lógica principal del sistema. (Primera entrega)

5.3 Persistencia de la información

- La persistencia es fundamental para el diseño, ya que el sistema deberá conservar la información incluso después de cerrar la aplicación; por esta razón, los datos necesarios para reconstruir el estado del club deberán guardarse en archivos y recuperarse cuando el programa vuelva a ejecutarse.

- Se desarrollará una estructura que permita separar la información de acuerdo con las necesidades del sistema; no es obligatorio utilizar un único archivo, por lo cual se podrá dividir la información siempre que se mantenga una organización clara y facilite la lectura y escritura de los datos.

- La persistencia deberá permitir recuperar usuarios, equipos, jugadores, competencias, reservas, entrenamientos, productos, inventarios, ventas y demás información necesaria para continuar utilizando la aplicación sin perder los registros creados anteriormente.

- Más adelante, dentro del diseño detallado, especificaremos los archivos utilizados, la información almacenada en cada uno, las clases responsables de su lectura y escritura y el momento en el que se realizarán las operaciones de carga y guardado.

6. DISEÑO GENERAL DE LA SOLUCIÓN

6.1. Organización del sistema

La aplicación se organizará alrededor de una clase principal denominada ClubDeportivoSeneca; esta clase tendrá el papel de coordinador general del sistema, por lo cual permitirá acceder a las principales operaciones y mantendrá comunicación con los distintos componentes de la solución, pero no será responsable de realizar por sí sola todas las validaciones o cálculos.

De esta manera, cuando una operación corresponda directamente a una entidad determinada, esa entidad será la encargada de resolverla. Por ejemplo, un equipo podrá controlar sus jugadores, una competencia mantendrá su información de inscripción y partidos, una venta manejará sus detalles y valores, mientras que una reserva conservará la información sobre la instalación, fecha, horario y modalidad. Esta decisión busca que la información y el comportamiento relacionado permanezcan juntos, en lugar de concentrar toda la lógica dentro de ClubDeportivoSeneca.

El sistema se dividirá en los siguientes grupos principales:

1. **Usuarios y administración** - encargado de socios, empleados, administrador, turnos y solicitudes de cambio.
2. **Deportes de conjunto y competencias** - donde se manejarán equipos, jugadores, categorías, partidos, competencias, nóminas, estadísticas y sanciones.
3. **Disciplinas individuales y entrenamiento** - donde se encontrarán las inscripciones por disciplina, niveles, rankings, instalaciones, reservas, partidos de práctica, sesiones y asistencias.
4. **Tienda, cafetería e inventario** - responsable de productos, ventas, descuentos, puntos, inventarios, sugerencias y mensualidades.
5. **Persistencia** - cuya responsabilidad será guardar y recuperar la información del sistema mediante archivos.
6. **Programas de prueba** - encargados únicamente de construir los diferentes escenarios y demostrar desde consola que las funcionalidades funcionan correctamente.

Estos grupos colaboran entre sí cuando una operación lo requiere. Por ejemplo, una compra puede involucrar un socio, un producto, una venta, el inventario y los puntos de fidelidad.

Además, cuando una clase deba manejar varios objetos del mismo tipo se utilizarán las colecciones de Java; de esta manera será posible almacenar, buscar, agregar o remover elementos sin construir estructuras adicionales innecesarias.

6.2 Organización del sistema

Se presentan los componentes de forma general; los atributos, métodos, tipos de datos y relaciones exactas se definirán en el UML detallado

Componente	Principales elementos	Función general
Coordinación general	ClubDeportivoSeneca	Mantener una visión general del sistema y coordinar operaciones que necesiten la colaboración de varios elementos
Usuarios y administración	Usuario, Socio, Empleado, Entrenador, Fisioterapeuta, Administrador, TurnoSemanal, SolicitudCambioTurno	Manejar usuarios, autenticación, información particular de cada tipo de usuario, turnos y solicitudes de cambio
Deportes de conjunto	Deporte, Categoria, Equipo, JugadorEquipo, Partido, PartidoInterno	Representar equipos, jugadores, categorías, partidos internos y sus reglas
Competencias	Competencia, InscripcionCompetencia, JugadorCompetencia, PartidoCompetencia	Manejar inscripción de equipos, nóminas, partidos oficiales, formatos y sanciones de cada competencia
Estadísticas de conjunto	EstadisticaJugador y estadísticas específicas de fútbol, baloncesto y voleibol	Mantener únicamente las estadísticas correspondientes a cada deporte
Disciplinas individuales	DisciplinaIndividual, InscripcionDisciplina, Instalacion, Reserva, PartidoPractica	Manejar nivel, ranking, instalaciones, reservas y partidos de práctica de tenis, pádel y tenis de mesa
Estadísticas individuales	EstadisticaPractica, EstadisticaTenisPadel, EstadisticaTenisMesa	Registrar los resultados y estadísticas propias de las disciplinas individuales

Entrenamientos	SesionEntrenamiento, Asistencia	Programar sesiones, manejar convocados y registrar asistencia y evaluación
Tienda y cafetería	Producto, ArticuloTienda, ProductoCafeteria, Bebida, Snack, Venta, DetalleVenta	Manejar productos y compras; además, aplicar impuestos, descuentos, propina y puntos
Inventario	UbicacionInventario, StockArticulo	Mantener la cantidad disponible de cada artículo según su ubicación y permitir movimientos de inventario
Administración comercial	SugerenciaProducto, PagoMensualidad	Registrar sugerencias de empleados y mantener la información necesaria para los reportes de mensualidades
Persistencia	Clases responsables de lectura y escritura	Guardar el estado de la aplicación en archivos y reconstruirlo al iniciar nuevamente el programa
Pruebas	Programas de prueba por escenario	Ejecutar desde consola los casos necesarios para demostrar los requerimientos y reglas de dominio

6.3 Distribución general de responsabilidades

Esta separación busca que cada clase tenga responsabilidades relacionadas entre sí y que no existan operaciones ubicadas en lugares donde no corresponden.

Elemento o grupo	Responsabilidad principal
ClubDeportivoSeneca	Coordinar operaciones generales, mantener acceso a los principales elementos del sistema y servir como punto central para las funcionalidades que requieren colaboración entre diferentes partes
Usuarios	Mantener la información de identificación y autenticación; cada tipo de usuario conserva además la información propia de su rol
Equipos y jugadores	Mantener integrantes, deporte, categoría, camiseta, estado y reglas relacionadas con la participación del jugador

Partidos y competencias	Mantener resultados, participantes, nóminas, fases, sanciones y demás información necesaria para competencias internas y externas
Disciplinas individuales	Mantener nivel y ranking por disciplina; además, controlar la información necesaria para reservas y partidos de práctica
Entrenamientos	Mantener sesiones, convocados, asistencia y evaluaciones
Ventas	Mantener los productos comprados, cantidades y valores; además, calcular los elementos que forman parte de una transacción
Inventario	Conocer las existencias y controlar los movimientos de artículos entre ubicaciones
Turnos	Mantener los horarios asignados y las solicitudes de cambio; las validaciones necesarias deberán asegurar que no se rompa la cobertura mínima
Persistencia	Leer y escribir la información en archivos; esta responsabilidad se mantendrá separada de la lógica deportiva, comercial y administrativa
Programa de prueba	Preparar datos, ejecutar operaciones y mostrar resultados; no deberán contener las reglas de negocio de la aplicación

Criterios que se seguirán durante el diseño detallado

Para mantener esta organización cuando se definan las clases y métodos, se utilizarán los siguientes criterios:

- **La información se manejará donde naturalmente pertenece** - por ejemplo, la camiseta y sanciones internas de un jugador pertenecen a su participación dentro de un equipo, mientras que las sanciones de una competencia pertenecen a su participación dentro de esa competencia.
- **La herencia solamente se utilizará cuando exista una relación "es un" real** - por esto, por ejemplo, Entrenador es un tipo de Empleado y PartidoCompetencia es un tipo de Partido, pero un Equipo no será tratado como un tipo de Partido. Este uso coincide con el concepto de herencia trabajado en clase.
- **Las relaciones uno a muchos se representarán mediante colecciones** - utilizando los tipos genéricos de Java vistos durante el curso; la estructura específica se elegirá según lo que necesite cada relación.
- **Las reglas de dominio se validarán en la lógica de la aplicación** - evitando que los programas de prueba tengan que decidir si una operación es válida o no; de esta manera, las mismas reglas funcionarán independientemente del programa desde el cual se invoquen.
- **Los errores de una operación se manejarán de manera explícita** - cuando corresponda, se utilizarán excepciones para informar que una operación no puede completarse, siguiendo el mecanismo visto en clase donde un método puede finalizar normalmente o lanzar una excepción que posteriormente será manejada por quien realizó la invocación.

- **La persistencia permanecerá separada de las clases del dominio** - las clases deportivas o comerciales no serán responsables de abrir y escribir directamente todos los archivos. El material de persistencia plantea precisamente que, además de diseñar los archivos, deben diseñarse las clases encargadas de leer y escribir la información.
- **Se evitará duplicar lógica** - una regla deberá tener un lugar claro donde se implemente y los demás elementos utilizarán esa operación cuando la necesiten.

7. DIAGRAMA UML DISEÑO DE ALTO NIVEL

Para representar la estructura completa del Club Deportivo Séneca se dividió el diseño de alto nivel en seis vistas; esta decisión permite mostrar todas las clases y relaciones relevantes sin concentrarlas en un único diagrama demasiado grande; cada vista se enfoca en un grupo de responsabilidades relacionado y, en conjunto, permiten comprender la estructura general del sistema.

7.1 Vista general

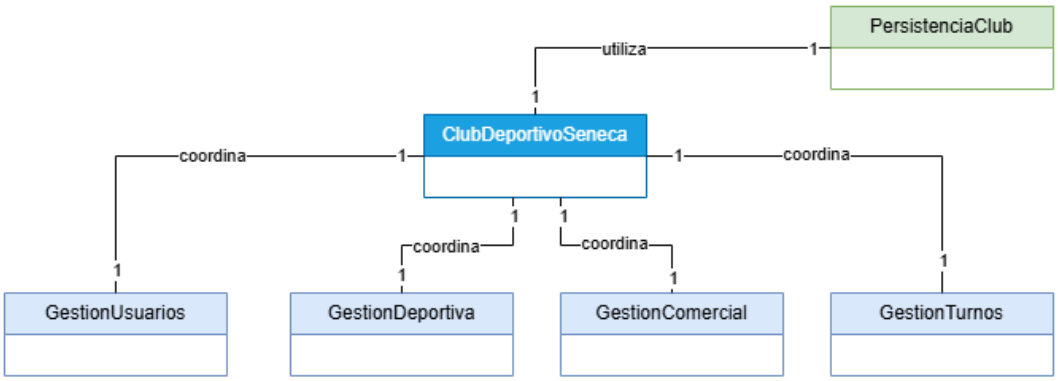


Imagen 1. UML de alto nivel – Vista general del sistema

Entidades principales:

- ClubDeportivoSeneca: representa el punto principal de coordinación de la aplicación.
- GestionUsuarios: concentra las operaciones relacionadas con usuarios.
- GestionDeportiva: agrupa la lógica deportiva.
- GestionComercial: administra tienda, cafetería e inventario.
- GestionTurnos: administra turnos y solicitudes de cambio.
- PersistenciaClub: se encarga de guardar y recuperar la información.

Relaciones y cardinalidades:

- ClubDeportivoSeneca 1 — 1 GestionUsuarios, relación coordina.
- ClubDeportivoSeneca 1 — 1 GestionDeportiva, relación coordina.
- ClubDeportivoSeneca 1 — 1 GestionComercial, relación coordina.
- ClubDeportivoSeneca 1 — 1 GestionTurnos, relación coordina.
- ClubDeportivoSeneca 1 — 1 PersistenciaClub, relación utiliza.

La cardinalidad 1 — 1 indica que una instancia de ClubDeportivoSeneca trabaja con una única instancia de cada gestor, donde cada uno de estos componentes pertenece a la coordinación de una única aplicación del

club. No se plantearon relaciones de muchos en esta vista porque estas clases representan componentes generales del sistema.

Se decidió hacer de esta manera para evitar que ClubDeportivoSeneca termine realizando autenticación, reservas, ventas, competencias, turnos, inventario y persistencia; en cambio, funciona como coordinador y delega cada área, lo que permite una distribución más clara de responsabilidades.

7.2 Usuarios y gestión de turnos

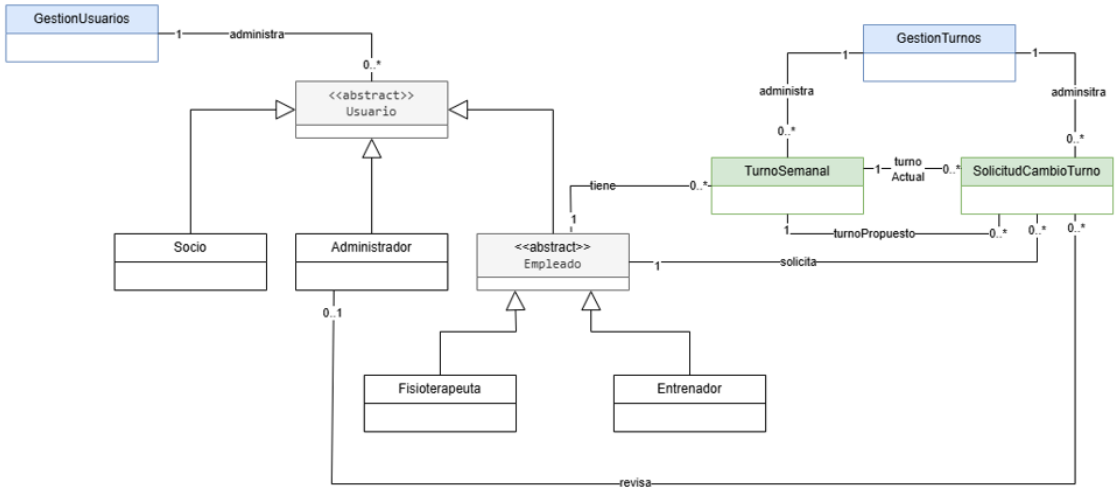


Imagen 2. UML de alto nivel – Usuario y gestión de turnos

Jerarquía de usuarios

Usuario funciona como clase general y abstracta; a partir de ella se especializan las siguientes entidades:

- Socio
- Empleado
- Administrador

Empleado también se definió como abstracta y se divide en:

- Entrenador
- Fisioterapeuta

Estas relaciones son de herencia, por lo cual no llevan cardinalidades.

Relaciones y cardinalidades:

- GestionUsuarios 1 — 0..* Usuario, administra: un gestor puede mantener cero o muchos usuarios; cada usuario pertenece a una gestión de usuarios.
- Empleado 1 — 0..* TurnoSemanal, tiene: un empleado puede tener varios turnos; cada turno corresponde a un solo empleado.
- Empleado 1 — 0..* SolicitudCambioTurno, solicita: un empleado puede realizar varias solicitudes; cada solicitud tiene un único solicitante.
- Administrador 0..1 — 0..* SolicitudCambioTurno, revisa: una solicitud pendiente puede no tener todavía administrador revisor y, una vez atendida, tendrá como máximo uno; un administrador puede revisar muchas solicitudes.

- GestionTurnos 1 — 0..* TurnoSemanal, administra.
- GestionTurnos 1 — 0..* SolicitudCambioTurno, administra.
- TurnoSemanal 1 — 0..* SolicitudCambioTurno, turnoActual
- TurnoSemanal 1 — 0..* SolicitudCambioTurno, turnoPropuesto.

Las dos últimas relaciones se separaron porque una solicitud debe conocer exactamente qué turno se desea modificar y cuál sería su reemplazo; cada solicitud referencia un único turnoActual y un único turnoPropuesto, mientras que un mismo turno puede aparecer en varias solicitudes a lo largo del tiempo.

La herencia permite compartir la información común de los usuarios sin duplicarla; adicionalmente, separar GestionUsuarios de GestionTurnos evita mezclar autenticación y administración de personas con la lógica propia de los horarios. Esta organización permite aplicar la regla de cobertura mínima sin convertir al Administrador en la clase que hace directamente todos los cálculos.

7.3 Deportes de conjunto y competencias

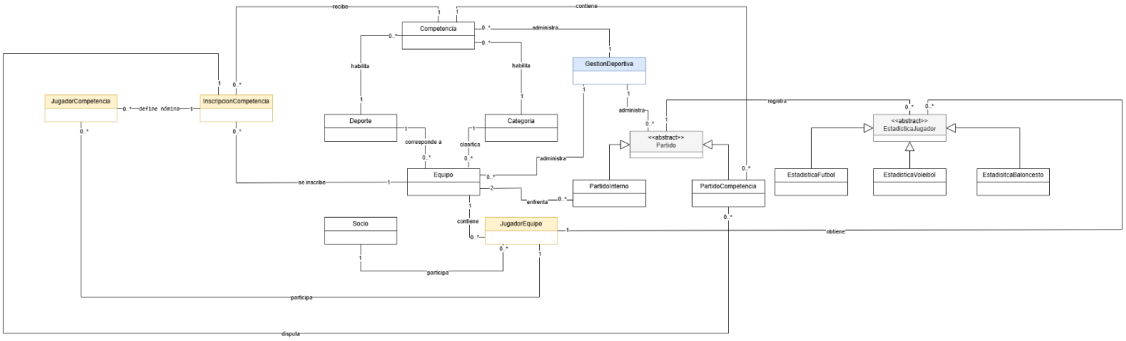


Imagen 3. UML de alto nivel – Deportes de conjunto y competencias

Equipos y participación de jugadores

Las entidades principales de este bloque son Deporte, Categoria, Equipo, Socio y JugadorEquipo.

Relaciones:

- Deporte 1 — 0..* Equipo, corresponde a.
- Categoria 1 — 0..* Equipo, clasifica.
- Equipo 1 — 0..* JugadorEquipo, contiene.
- Socio 1 — 0..* JugadorEquipo, participa.

Un equipo pertenece exactamente a un deporte y una categoría, sin embargo, cada deporte y cada categoría pueden tener diferentes equipos. JugadorEquipo se creó como una clase intermedia porque la participación de un socio en un equipo posee información propia, por ejemplo posición, número de camiseta y sanciones internas; estos datos no pertenecen únicamente a Socio ni únicamente a Equipo.

Partidos:

Partido se plantea como una clase abstracta y tiene dos especializaciones. Al ser herencia, estas dos conexiones no tienen cardinalidades:

- PartidoInterno
- PartidoCompetencia

Competencias:

- Deporte 1 — 0..* Competencia, habilita.
- Categoria 1 — 0..* Competencia, habilita.
- Competencia 1 — 0..* InscripcionCompetencia, recibe.
- Equipo 1 — 0..* InscripcionCompetencia, se inscribe.
- InscripcionCompetencia 1 — 0..* JugadorCompetencia, define nómina.
- JugadorEquipo 1 — 0..* JugadorCompetencia, participa.
- Competencia 1 — 0..* PartidoCompetencia, contiene.
- InscripcionCompetencia 1 — 0..* PartidoCompetencia, disputa.

La separación entre Equipo e InscripcionCompetencia permite que un mismo equipo participe en distintas competencias a lo largo del tiempo; asimismo, JugadorCompetencia permite mantener una nómina específica y conservar sanciones propias de esa competencia. Esto representa la regla según la cual solo los jugadores pertenecientes a la nómina pueden participar en los partidos oficiales.

Estadísticas:

EstadisticaJugador es abstracta y se especializa en:

- EstadisticaFutbol

- EstadisticaBaloncesto

- EstadisticaVoleibol

Relaciones:

- Partido 1 — 0..* EstadisticaJugador, registra.

- JugadorEquipo 1 — 0..* EstadisticaJugador, obtiene.

Cada estadística corresponde a un partido y a un jugador determinado; un partido puede generar varias estadísticas y un jugador puede acumular registros en diferentes encuentros.

Finalmente, GestionDeportiva mantiene relaciones 1 — 0..* con Equipo, Competencia y Partido; de esta manera, el gestor puede administrar las colecciones principales sin tener que relacionarse directamente con absolutamente todas las clases del bloque.

7.4 Disciplinas individuales y entrenamiento

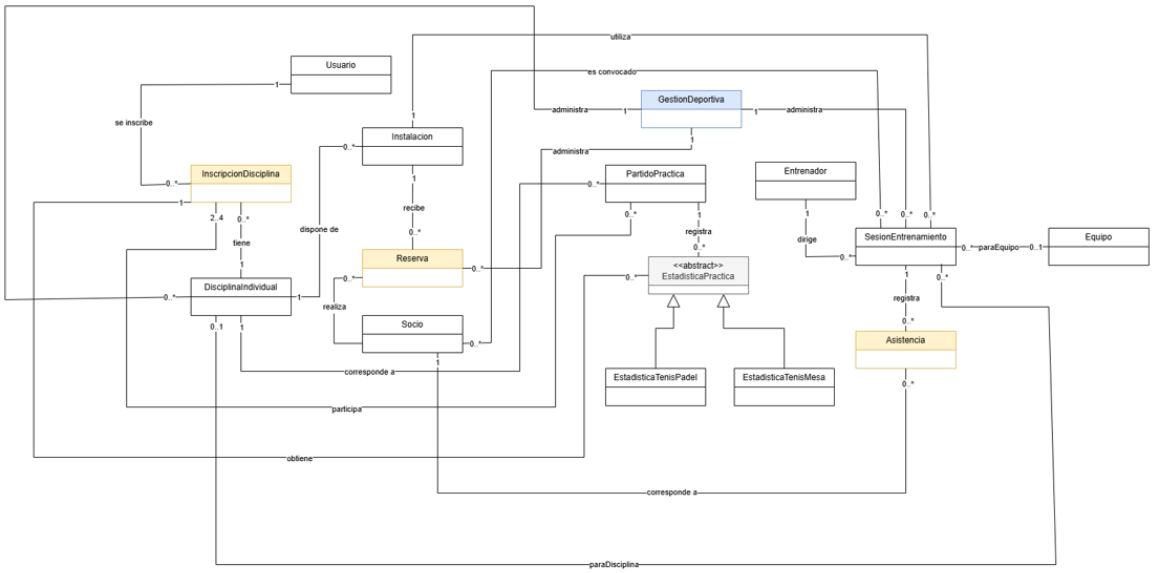


Imagen 4. UML de alto nivel – Disciplinas individuales y entrenamiento

Disciplinas e inscripciones

- DisciplinaIndividual 1 — 0..* InscripcionDisciplina, tiene.
- Usuario 1 — 0..* InscripcionDisciplina, se inscribe.

Cada inscripción corresponde a una sola disciplina y a un solo titular, sin embargo, una disciplina puede tener múltiples participantes y un usuario puede encontrarse inscrito en distintas disciplinas. Se utilizó Usuario porque la participación puede corresponder a un socio o, en los casos permitidos, a un entrenador.

Instalaciones y reservas

- DisciplinaIndividual 1 — 0..* Instalacion, dispone de.
- Socio 1 — 0..* Reserva, realiza.
- Instalacion 1 — 0..* Reserva, recibe.

Cada instalación pertenece a una disciplina determinada, una instalación puede recibir muchas reservas en momentos diferentes, mientras que cada reserva identifica una única instalación y un único socio.

Partidos de práctica y estadísticas

- DisciplinaIndividual 1 — 0..* PartidoPractica, corresponde a.
- InscripcionDisciplina 2..4 — 0..* PartidoPractica, participa.
- PartidoPractica 1 — 0..* EstadisticaPractica, registra.
- InscripcionDisciplina 1 — 0..* EstadisticaPractica, obtiene.

La multiplicidad 2..4 expresa que un partido de práctica debe tener entre dos y cuatro participantes, dependiendo de la disciplina y modalidad. EstadisticaPractica se definió como abstracta y se especializa en EstadisticaTenisPadel y EstadisticaTenisMesa, permitiendo que solo se almacenan estadísticas correspondientes a cada disciplina.

Las reglas del análisis establecen que los jugadores deben tener niveles iguales o adyacentes, salvo la excepción cuando alguno también sea entrenador. También, después de los partidos debe actualizarse el ranking correspondiente.

- Entrenador 1 — 0..* SesionEntrenamiento, dirige.
- Instalacion 1 — 0..* SesionEntrenamiento, utiliza.
- SesionEntrenamiento 0..* — 0..* Socio, convoca.
- SesionEntrenamiento 1 — 0..* Asistencia, registra.
- Socio 1 — 0..* Asistencia, corresponde a.
- Equipo 0..1 — 0..* SesionEntrenamiento, paraEquipo.
- DisciplinaIndividual 0..1 — 0..* SesionEntrenamiento, paraDisciplina.

La relación muchos a muchos entre sesiones y socios representa la lista de convocados; en cambio, Asistencia se utiliza para conservar el resultado de la participación, como asistió, faltó o llegó tarde. Se decidió mantener ambas relaciones porque ser convocado y registrar asistencia son momentos diferentes del proceso; de acuerdo con las reglas del proyecto, solamente los jugadores convocados pueden tener asistencia registrada.

7.5 Tienda, cafetería e inventario

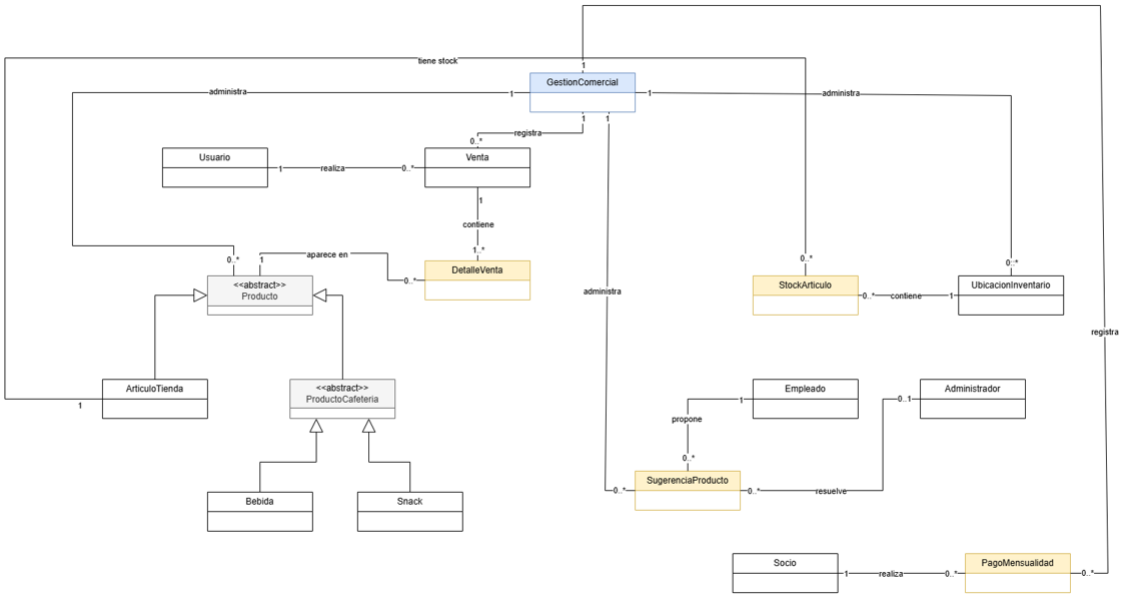


Imagen 5. UML de alto nivel – Tienda, cafetería e inventario

Productos

Producto es abstracta y se especializa en:

- ArticuloTienda
- ProductoCafeteria

ProductoCafeteria es abstracta y tiene como subclases:

- Bebida
- Snack

Estas relaciones son de herencia y no utilizan cardinalidad; se decidió separarlas porque tienda y cafetería manejan reglas diferentes de impuestos y características propias.

Ventas

- Usuario 1 — 0..* Venta, realiza.
- Venta 1 — 1..* DetalleVenta, contiene.
- Producto 1 — 0..* DetalleVenta, aparece en.

DetalleVenta evita una relación directa muchos a muchos entre ventas y productos; además, permite conservar posteriormente cantidad y precio unitario de la operación.

Inventario

- ArticuloTienda 1 — 0..* StockArticulo, tiene stock.
- UbicacionInventario 1 — 0..* StockArticulo, contiene.

StockArticulo representa una combinación concreta de artículo, ubicación y cantidad, lo que permite que un mismo artículo tenga existencias separadas en diferentes lugares y hace posible mover inventario sin duplicar el producto.

Sugerencias y mensualidades

- Empleado 1 — 0..* SugerenciaProducto, propone.
- Administrador 0..1 — 0..* SugerenciaProducto, resuelve.
- Socio 1 — 0..* PagoMensualidad, realiza.

Una sugerencia recién creada puede no tener administrador revisor, por eso se utiliza 0..1; una vez revisada tendrá como máximo uno, mientras que un administrador puede resolver diferentes sugerencias. PagoMensualidad se mantiene separado de Venta, ya que representa otro rubro financiero y permitirá que los reportes distingan entre tienda, cafetería y mensualidades.

Gestión comercial

GestionComercial mantiene relaciones 1 — 0..* con:

- Producto, administra.
- Venta, registra.
- UbicacionInventario, administra.
- SugerenciaProducto, administra.
- PagoMensualidad, registra.

Con esto el módulo mantiene el estado comercial completo sin trasladar esta responsabilidad a ClubDeportivoSeneca.

7.6 Persistencia y programas de prueba

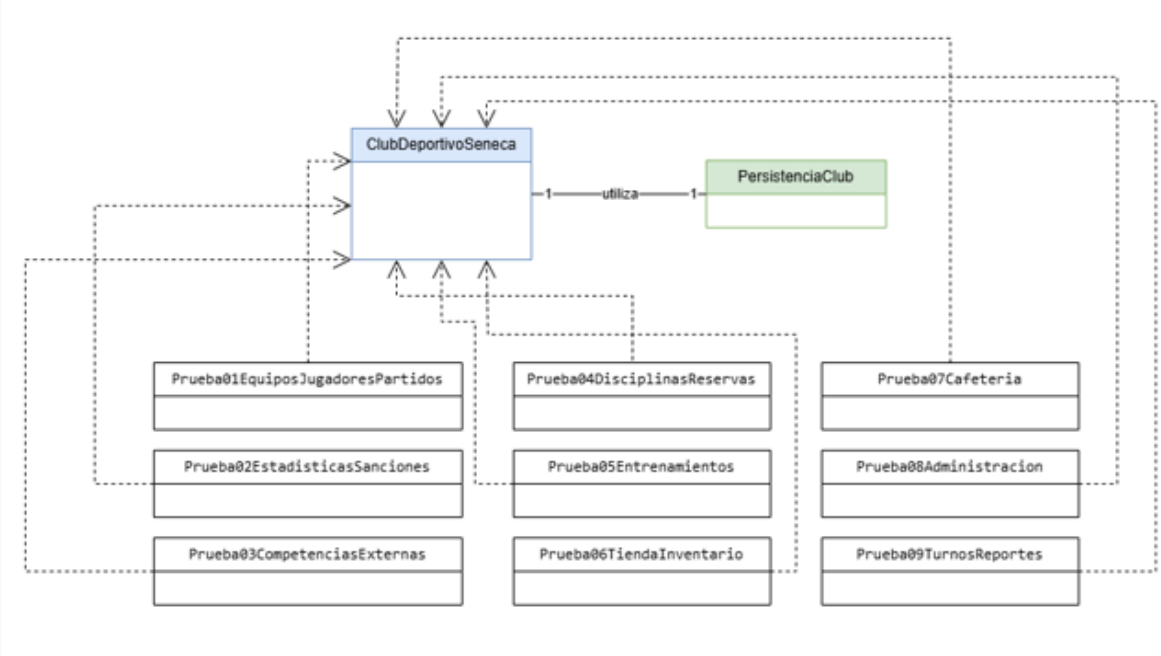


Imagen 6. UML de alto nivel – Persistencia y programas de pruebas

Las dos clases principales de esta vista son ClubDeportivoSeneca y PersistenciaClub:

- ClubDeportivoSeneca 1 — 1 PersistenciaClub, utiliza.

Se mantuvo la misma cardinalidad definida en la vista general, debido a que una aplicación utiliza un único componente responsable de guardar y recuperar su información.

Las nueve clases Prueba01 hasta Prueba09 se conectan mediante dependencias hacia ClubDeportivoSeneca; estas líneas no llevan cardinalidad porque los programas de prueba no forman parte del estado permanente de la aplicación, simplemente la utilizan para ejecutar escenarios.

Cada prueba corresponde a los casos definidos desde el análisis: equipos y jugadores, estadísticas y sanciones, competencias, disciplinas y reservas, entrenamientos, tienda, cafetería, administración y turnos/reportes.

8. DIAGRAMA UML DETALLADO

Los UML detallados mantienen la misma estructura, relaciones y multiplicidades del punto 7; sin embargo, se añaden los atributos y métodos, en los que se incluyen algunos constructores.

8.1. Vista general

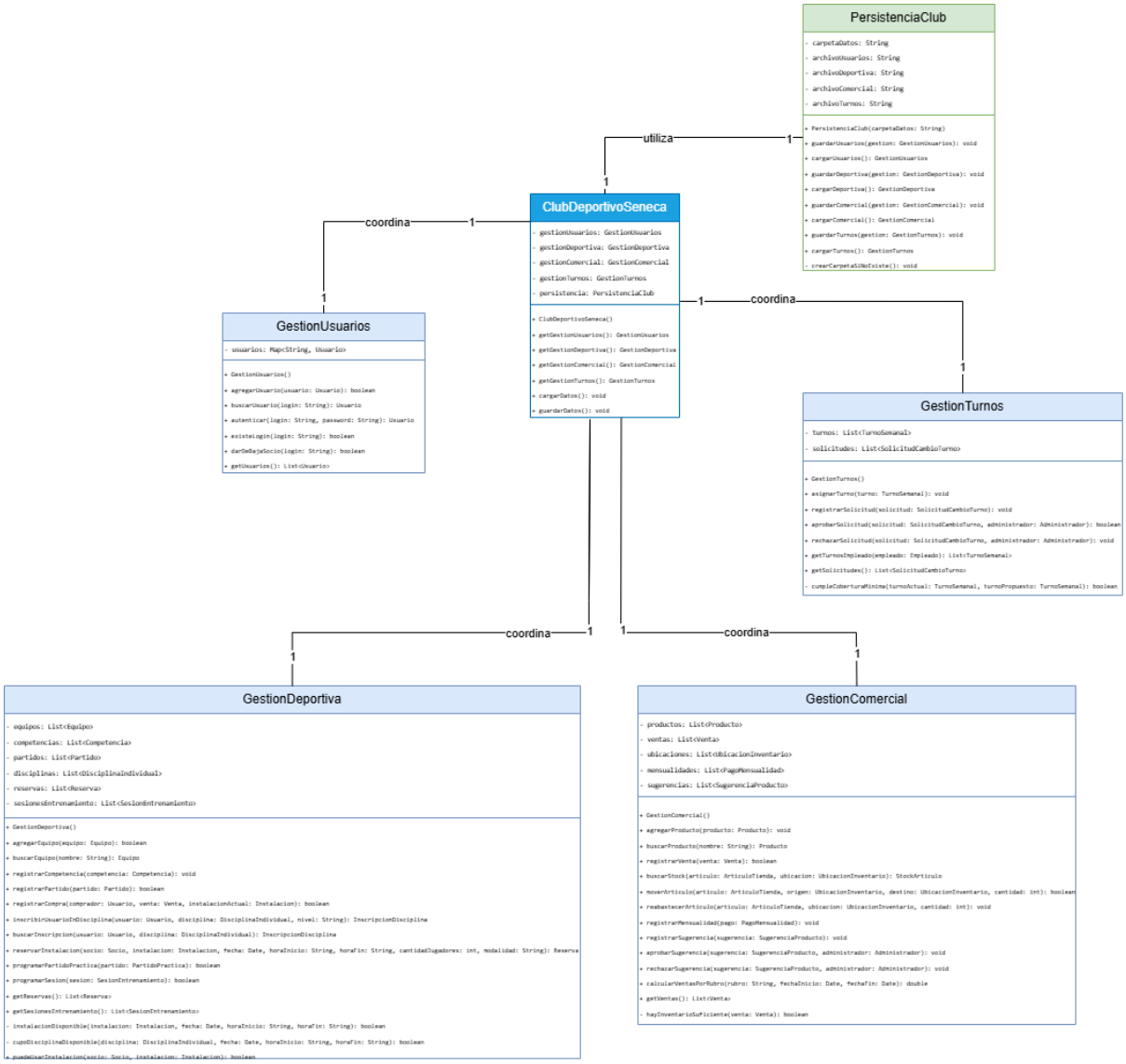


Imagen 7. UML Detallado – Vista General

ClubDeportivoSeneca

Mantiene como atributos privados:

- gestionUsuarios: GestionUsuarios
- gestionDeportiva: GestionDeportiva
- gestionComercial: GestionComercial
- gestionTurnos: GestionTurnos
- persistencia: PersistenciaClub

Estos atributos representan las asociaciones 1 — 1 observadas en el UML; la clase principal mantiene una referencia a cada gestor y al componente de persistencia, de manera que puede coordinar las operaciones sin almacenar directamente todas las colecciones del sistema.

Sus métodos públicos permiten acceder a los gestores mediante operaciones como getGestionUsuarios(), getGestionDeportiva(), getGestionComercial() y getGestionTurnos(), además de cargarDatos() y guardarDatos() que centralizan el acceso a las operaciones de persistencia.

Gestores

- GestionUsuarios mantiene un Map<String, Usuario>; el login se utiliza como llave para facilitar la búsqueda de los usuarios. Sus métodos permiten agregar, buscar, autenticar y dar de baja usuarios.

- GestionDeportiva conserva colecciones de equipos, competencias, partidos, disciplinas, reservas y sesiones de entrenamiento; además, ofrece operaciones para registrar inscripciones, programar partidos, gestionar reservas y comprobar las condiciones necesarias para las actividades deportivas.

- GestionComercial mantiene colecciones de productos, ventas, ubicaciones de inventario, mensualidades y sugerencias; sus métodos permiten registrar ventas, consultar existencias, trasladar artículos, registrar pagos y aprobar o rechazar sugerencias.

- GestionTurnos mantiene listas de turnos semanales y solicitudes de cambio; sus operaciones permiten asignar turnos, registrar solicitudes, aprobarlas o rechazarlas y comprobar que se conserve la cobertura mínima del personal.

- PersistenciaClub mantiene las rutas de los archivos destinados a guardar la información de cada componente; sus métodos permiten guardar y cargar los datos correspondientes.

Separamos estas responsabilidades para evitar que la clase principal concentre toda la lógica; asimismo, los gestores pueden mantener sus propias colecciones y ejecutar las operaciones correspondientes a cada área.

Cardinalidades

Se conservan las relaciones 1 — 1 entre ClubDeportivoSeneca y los cuatro gestores, además de PersistenciaClub; esto representa una instancia de cada componente asociada con la clase principal en el diseño propuesto.

8.2. Usuarios y gestión de turnos

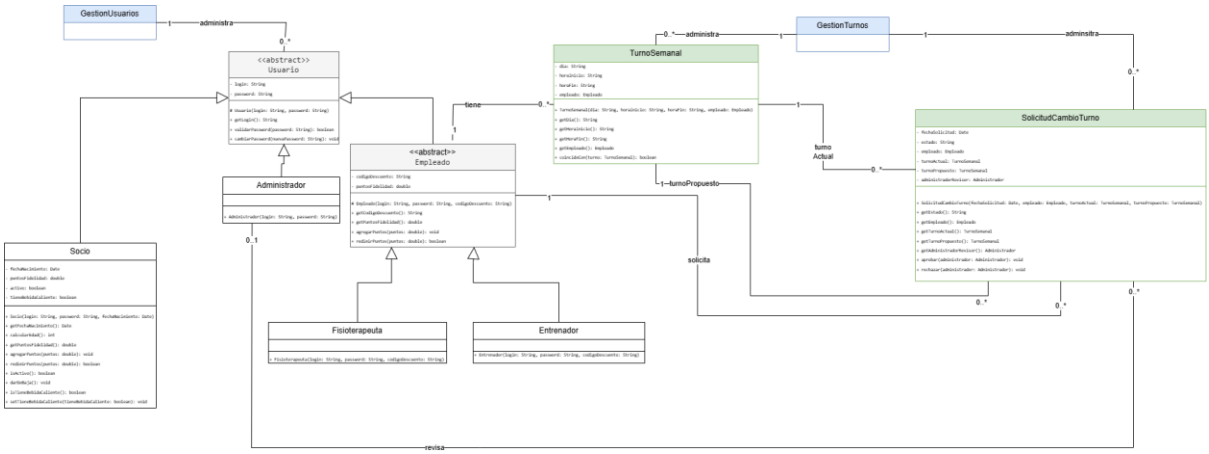


Imagen 8. UML Detallado – Usuarios y gestión de turnos

Usuario

Es una clase abstracta que reúne los datos y operaciones comunes de las personas que acceden al sistema. Mantiene los atributos privados:

- login: String
- password: String

Sus métodos públicos incluyen getLogin(), validarPassword(password: String) y cambiarPassword(nuevaPassword: String), además del constructor correspondiente.
Al ser abstracta, sirve como base para definir usuarios concretos sin tener que repetir la información de acceso.

Socio

Hereda de Usuario y añade atributos relacionados con el socio:
- fechaNacimiento: Date
- puntosFidelidad: double
- activo: boolean
- tieneBebidaCaliente: boolean

Sus métodos permiten consultar la fecha de nacimiento, calcular la edad, administrar los puntos de fidelidad, activar o desactivar al socio y consultar o actualizar si tiene una bebida caliente. Esta última información permite representar una condición relevante para las restricciones de uso de algunas instalaciones.

Empleado

Es una clase abstracta que hereda de Usuario y contiene los atributos codigoDescuento: String y puntosFidelidad: double. Sus operaciones permiten consultar el código de descuento y los puntos acumulados, además de agregar o redimir puntos.

De esta clase heredan Entrenador y Fisioterapeuta, que representan los tipos de empleados contemplados en el sistema; Administrador, por su parte, hereda directamente de Usuario y representa al responsable de revisar solicitudes de cambio de turno.

TurnoSemanal

Mantiene los atributos privados:
- dia: String
- horaInicio: String
- horaFin: String
- empleado: Empleado

Sus métodos permiten consultar estos valores y comprobar si dos turnos coinciden.

La clase representa un turno asignado a un empleado; por eso, concentra la información necesaria para identificar el día y el intervalo de trabajo, facilitando las consultas y las comprobaciones de cobertura.

SolicitudCambioTurno

Contiene los atributos:
- fechaSolicitud: Date
- estado: String
- empleado: Empleado
- turnoActual: TurnoSemanal
- turnoPropuesto: TurnoSemanal
- administradorRevisor: Administrador

Sus métodos permiten consultar el estado de la solicitud, identificar al empleado y los turnos involucrados, y aprobar o rechazar el cambio mediante las operaciones correspondientes. De esta manera, el sistema conserva la información necesaria para evaluar una modificación sin confundirla con la asignación original.

Cardinalidades y relaciones

- Usuario – GestionUsuarios: el gestor administra 0..* usuarios; cada usuario se relaciona con una instancia del gestor.
- Usuario – Socio y Empleado: las flechas de generalización indican herencia; las clases especializadas reciben los atributos y métodos comunes de Usuario.
- Empleado – TurnoSemanal: cada turno corresponde a un empleado y un empleado puede tener varios turnos.
- GestionTurnos – TurnoSemanal: el gestor administra 0..* turnos.
- GestionTurnos – SolicitudCambioTurno: el gestor puede mantener 0..* solicitudes.
- SolicitudCambioTurno – Administrador: la solicitud contempla un administrador revisor opcional, según la multiplicidad representada en el diagrama.

Decidimos separar los datos de acceso, la información laboral y las solicitudes de cambio porque cada elemento cumple una función distinta; asimismo, la estructura permite verificar la cobertura de entrenadores y fisioterapeutas antes de aprobar un cambio, sin mezclar esta regla con la autenticación de usuarios.

8.3. Deportes de conjunto y competencias

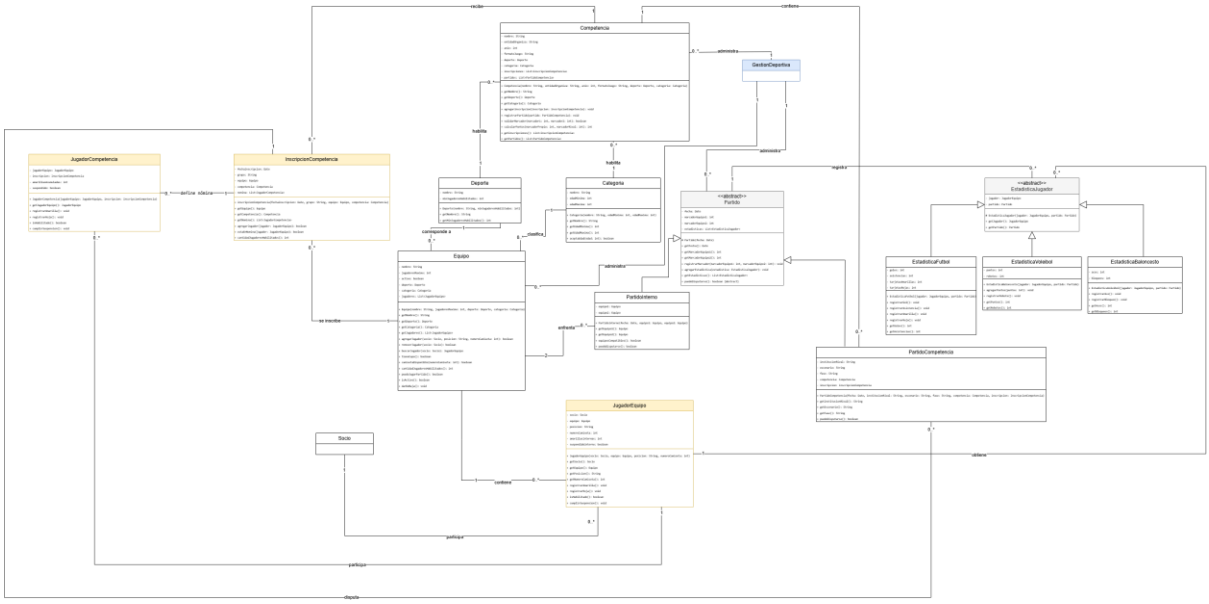


Imagen 9. UML Detallado – Deportes de conjunto y competencias

Deporte y Categoria

La clase Deporte representa la disciplina deportiva a la que pertenece un equipo, mientras que Categoria permite organizar los equipos según los criterios establecidos, como los límites de edad; de esta manera, ambas clases ayudan a comprobar las condiciones necesarias antes de registrar una participación deportiva.

Atributos principales de Deporte:

- nombre: String
- minJugadoresHabilitados: int

Atributos principales de Categoria:

- nombre: String
- edadMinima: int
- edadMaxima: int

Sus métodos permiten consultar la información y validar las condiciones correspondientes; asimismo, la categoría facilita comprobar si los jugadores cumplen los requisitos de edad antes de incorporarlos a una competición.

Equipo

Mantiene los datos necesarios para identificar al equipo, controlar su composición y comprobar si puede participar en un encuentro.

Atributos:

- nombre: String
- jugadoresMaximo: int
- tieneSuspendidos: boolean
- deporte: Deporte
- categoria: Categoria
- jugadores: List<JugadorEquipo>

Sus métodos permiten consultar y modificar los datos, agregar o retirar jugadores, comprobar el cupo disponible, validar los números de camiseta y verificar si el equipo cuenta con suficientes jugadores habilitados; por lo tanto, Equipo concentra las operaciones relacionadas con la composición de la plantilla.

JugadorEquipo

Esta clase representa la vinculación de un socio con un equipo; decidimos mantenerla separada de Socio porque un socio puede participar en diferentes deportes y tener información específica en cada equipo.

Atributos principales:

- jugador: Socio
- equipo: Equipo
- posicion: String
- numeroCamiseta: int
- suspendido: boolean

Sus métodos permiten consultar la información y administrar las condiciones relacionadas con la participación del jugador; así, se puede comprobar si está habilitado para jugar y si su número de camiseta cumple las reglas del equipo.

Competencia e InscripcionCompetencia

La clase Competencia representa una competición deportiva y conserva los datos necesarios para identificarla, organizarla y gestionar sus participantes.

Atributos principales de Competencia:

- nombre: String
- entidadOrganizadora: String
- anio: int
- formatoJuego: String

- deporte: Deporte
- categoria: Categoria
- inscripciones: List<InscripcionCompetencia>

Por otro lado, InscripcionCompetencia representa el registro de un equipo en una competición.

Atributos principales de InscripcionCompetencia:

- fechaInscripcion: Date
- estado: String
- equipo: Equipo
- competencia: Competencia
- observaciones: String

Los métodos de ambas clases permiten gestionar las inscripciones y comprobar las condiciones de participación; de esta manera, el sistema puede verificar que el equipo corresponda a la categoría y al deporte de la competición antes de confirmar el registro.

Partidos y estadísticas

Las clases relacionadas con los partidos permiten representar los encuentros deportivos y registrar sus resultados; además, la especialización facilita diferenciar los partidos internos de los que se disputan en una competición.

La clase abstracta Partido reúne los datos y operaciones comunes de un encuentro, mientras que PartidoInterno y PartidoCompetencia representan modalidades específicas.

Atributos principales de Partido:

- fecha: Date
- resultado: String
- equipoLocal: Equipo
- equipoVisitante: Equipo

Atributos principales de PartidoInterno:

- ubicacion: String
- estado: String

Atributos principales de PartidoCompetencia:

- identificador: String
- competencia: Competencia
- inscripcion: InscripcionCompetencia

La clase abstracta EstadisticaJugador reúne las operaciones comunes para registrar estadísticas; a su vez, EstadisticaFutbol, EstadisticaVoleibol y EstadisticaBaloncesto permiten almacenar información propia de cada deporte, evitando tratar todas las estadísticas como si fueran iguales.

Cardinalidades y relaciones

- Deporte – Equipo: un deporte puede estar asociado con 0..* equipos, mientras que cada equipo corresponde a un deporte.

- Categoria – Equipo: una categoría puede clasificar varios equipos; cada equipo pertenece a una categoría.
- Equipo – JugadorEquipo: un equipo puede tener varios registros de jugadores (0..*), cada uno vinculado con su equipo correspondiente.
- Competencia – InscripcionCompetencia: una competición puede tener varias inscripciones (0..*), cada una asociada con la competición correspondiente.
- Equipo – PartidoInterno: los equipos participan en encuentros internos; las multiplicidades del diagrama representan cuántos equipos intervienen en cada partido.
- Partido – sus especializaciones: las flechas de generalización representan herencia; las clases especializadas reciben los elementos comunes de Partido.
- EstadisticaJugador – sus especializaciones: las estadísticas específicas heredan las operaciones comunes de la clase abstracta.

Con esta distribución separamos la información de los equipos, las inscripciones, los partidos y las estadísticas; asimismo, logramos que cada clase tenga una responsabilidad concreta y que las validaciones deportivas se realicen antes de confirmar una participación.

8.4 Disciplinas individuales

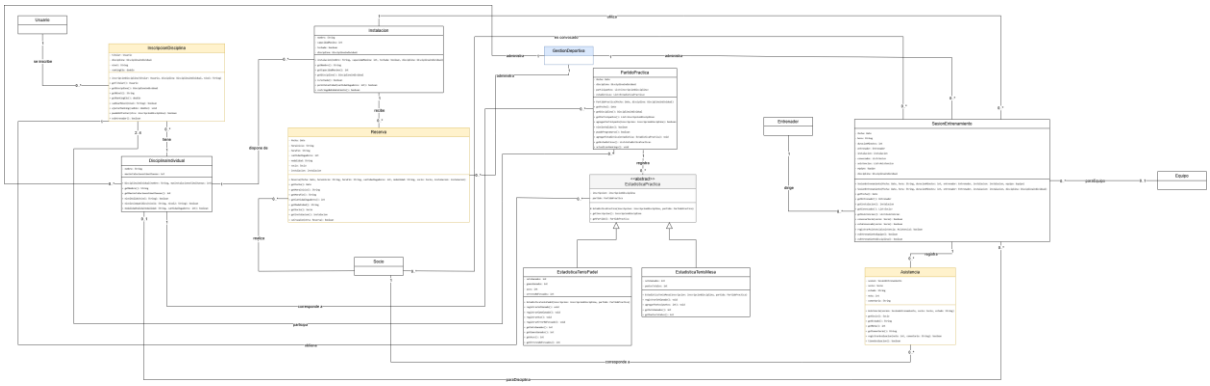


Imagen 10. UML Detallado – Disciplinas individuales

DisciplinaIndividual

Representa las disciplinas individuales que ofrece el club; sus atributos permiten identificar la disciplina y administrar las inscripciones de los socios, mientras que sus métodos facilitan las operaciones relacionadas con la participación y la organización de las actividades.

Atributos principales:

- nombre: String
- sesionesEntrenamiento: List<SesionEntrenamiento>

Sus métodos permiten consultar los datos de la disciplina y gestionar las inscripciones correspondientes; de esta manera, el sistema puede organizar las actividades individuales sin depender de la estructura de los deportes de conjunto.

InscripcionDisciplina

Esta clase representa el registro de un usuario en una disciplina individual; se mantiene separada de Usuario porque una inscripción tiene información propia, como el nivel del participante, la fecha del registro y su estado.

Atributos:

- usuario: Usuario
- disciplina: DisciplinaIndividual
- nivel: String
- fecha: Date
- estado: String

Sus métodos permiten consultar la información y administrar el proceso de inscripción; asimismo, el nivel del participante puede utilizarse para comprobar las condiciones necesarias al organizar partidos de práctica.

Instalacion

Representa los espacios físicos que utiliza el club para realizar las actividades deportivas; contiene información para identificar cada instalación y comprobar si se puede utilizar en una fecha y un horario determinados.

Atributos:

- nombre: String
- capacidadMaxima: int
- disciplina: DisciplinaIndividual
- estado: String
- reservas: List<Reserva>

Sus métodos permiten consultar los datos de la instalación y comprobar las condiciones de disponibilidad; por esto, la clase ayuda a evitar reservas que no cumplan los requisitos del espacio.

Reserva
Mantiene la información necesaria para registrar la solicitud de uso de una instalación; además, permite identificar al socio, el horario y las condiciones de la actividad.

Atributos:
- fecha: Date
- horarioInicio: String
- horarioFin: String
- cantidadJugadores: int
- modalidad: String
- socio: Socio
- instalacion: Instalacion

Sus métodos permiten consultar y validar la reserva, comprobar la disponibilidad de la instalación y verificar que la cantidad de jugadores sea compatible con su capacidad; de esta manera, se puede controlar el uso de los espacios deportivos.

PartidoPractica y estadísticas

PartidoPractica representa los encuentros organizados entre participantes de disciplinas individuales; permite registrar los datos del partido y comprobar las condiciones necesarias para su realización.

Atributos principales de PartidoPractica:

- fecha: Date
- disciplina: DisciplinaIndividual

- participantes: List<InscripcionDisciplina>
- estadisticas: List<EstadisticaPractica>

La clase abstracta EstadisticaPractica reúne las operaciones comunes para registrar estadísticas de los encuentros; a su vez, EstadisticaTenisPadel y EstadisticaTenisMesa permiten representar los datos particulares de cada disciplina.

Esta separación evita duplicar las operaciones generales de registro y, al mismo tiempo, permite adaptar las estadísticas a las características de cada deporte.

SesionEntrenamiento y Asistencia

SesionEntrenamiento representa una actividad programada con un entrenador; permite organizar la fecha, el horario, la instalación y los participantes convocados.

Atributos principales de SesionEntrenamiento:

- fecha: Date
- hora: String
- horaFin: String
- entrenador: Entrenador
- asistentes: List<Asistencia>
- instalacion: Instalacion
- convocatoria: List<Socio>

Sus métodos permiten consultar los datos de la sesión, gestionar las convocatorias y registrar la asistencia; de esta manera, se puede mantener un seguimiento de las actividades programadas.

Por otro lado, Asistencia representa el registro de participación de un socio en una sesión.

Atributos:

- sesion: SesionEntrenamiento
- socio: Socio
- estado: String
- fecha: Date
- comentario: String

Los métodos de esta clase permiten consultar la información y registrar o modificar el estado de asistencia; asimismo, los comentarios pueden utilizarse para documentar observaciones relacionadas con la sesión.

Cardinalidades y relaciones

- Usuario – InscripcionDisciplina: un usuario puede tener varias inscripciones (0..*); cada inscripción corresponde al usuario registrado.
- DisciplinaIndividual – InscripcionDisciplina: una disciplina puede tener múltiples inscripciones; cada registro identifica la disciplina correspondiente.
- Instalacion – Reserva: una instalación puede recibir varias reservas (0..*), mientras que cada reserva identifica la instalación solicitada.
- Socio – Reserva: un socio puede realizar varias reservas; cada reserva corresponde al socio que la solicita.

- PartidoPractica – EstadisticaPractica: un partido puede tener varios registros estadísticos, de acuerdo con las multiplicidades del diagrama.
- EstadisticaPractica – sus especializaciones: las flechas de generalización indican que las estadísticas específicas heredan las operaciones comunes de la clase abstracta.
- SesionEntrenamiento – Asistencia: una sesión puede tener múltiples registros de asistencia; cada registro identifica la sesión y el socio correspondiente.
- Entrenador – SesionEntrenamiento: el entrenador dirige las sesiones que tiene asignadas; las multiplicidades indican cuántas sesiones puede dirigir y cuántos entrenadores se asocian con cada una.

En conjunto, estas clases permiten organizar las disciplinas individuales, las reservas y los entrenamientos de forma independiente; además, logramos separar la planificación de las actividades del seguimiento de los participantes, facilitando las validaciones de disponibilidad y el registro de asistencia.

8.5 Tienda, cafetería e inventario

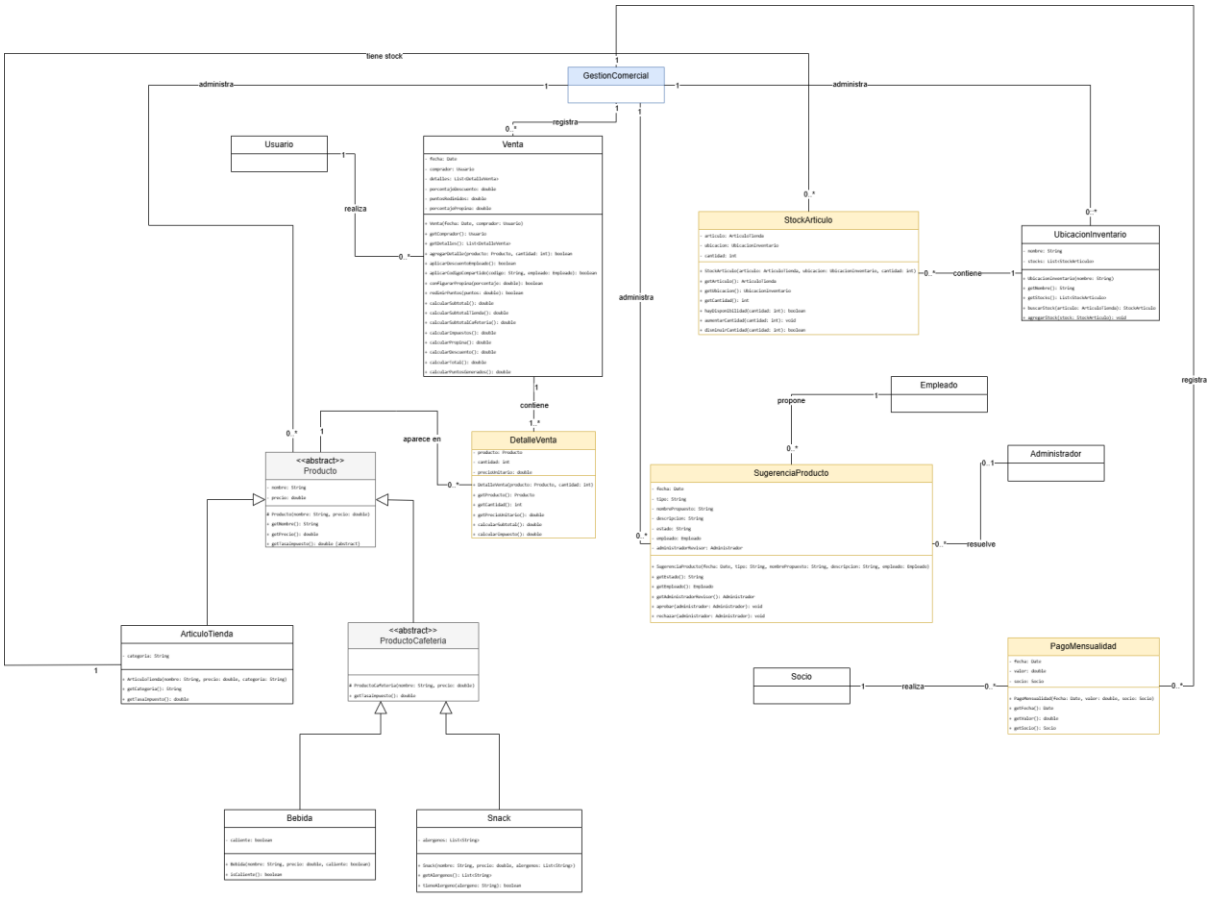


Imagen 11. UML Detallado – Tienda, cafetería e inventario

Producto

Es una clase abstracta que reúne las características comunes de los productos comercializados por el club; de esta manera, los productos de la tienda y de la cafetería comparten una estructura básica, pero pueden incorporar atributos y operaciones específicos.

Atributos:

- nombre: String
- precio: double

Sus métodos permiten consultar el nombre y el precio, además de calcular el impuesto correspondiente mediante una operación abstracta que se concreta en las clases especializadas.

ArticuloTienda

Representa los artículos que se venden en la tienda deportiva; hereda los atributos comunes de Producto e incorpora la información necesaria para identificar su categoría.

Atributos:

- categoria: String

Sus métodos permiten consultar la categoría y calcular el impuesto correspondiente; por lo tanto, esta clase mantiene las particularidades de los artículos deportivos sin duplicar los datos generales del producto.

ProductoCafeteria, Bebida y Snack

ProductoCafeteria es una clase abstracta que representa los productos de la cafetería; permite definir la estructura común de esta categoría y sirve como base para los productos concretos.

Bebida incorpora el atributo:

- caliente: boolean

Este atributo permite identificar si la bebida es caliente; a su vez, Snack incorpora la información de los alérgenos.

Atributos de Snack:

- alergenos: List<String>

Los métodos permiten consultar las características de cada producto y aplicar las comprobaciones correspondientes; asimismo, registrar los alérgenos facilita consultar la información necesaria antes de completar una venta.

Venta

Representa una transacción comercial realizada en el club; conserva los datos del comprador y los detalles de los productos, además de la información necesaria para calcular el importe final.

Atributos:

- fecha: Date
- comprador: Usuario
- detalles: List<DetalleVenta>
- porcentajeDescuento: double
- puntosFidelidad: double
- porcentajePropina: double

Sus métodos permiten agregar productos, aplicar descuentos, configurar la propina, redimir puntos de fidelidad y calcular los subtotales, impuestos y total de la compra; por esto, Venta concentra las operaciones necesarias para obtener el valor final de cada transacción.

DetalleVenta

Representa cada línea de producto incluida en una venta; decidimos mantener esta clase separada porque una misma transacción puede contener varios productos y cantidades diferentes.

Atributos:

- producto: Producto
- cantidad: int
- precioUnitario: double

Sus métodos permiten consultar el producto, la cantidad y el precio unitario, además de calcular el subtotal y los impuestos correspondientes; de esta manera, cada detalle conserva la información necesaria para calcular correctamente el valor de la compra.

StockArticulo y UbicacionInventario

StockArticulo representa la cantidad disponible de un artículo en una ubicación determinada; esta separación permite controlar las existencias sin confundir la información comercial del producto con su disponibilidad física.

Atributos de StockArticulo:

- articulo: ArticuloTienda
- ubicacion: UbicacionInventario
- cantidad: int

Sus métodos permiten consultar el artículo y la ubicación, verificar las existencias, aumentar o disminuir cantidades y comprobar si hay stock suficiente.

UbicacionInventario representa el lugar donde se almacenan los productos.

Atributos:

- nombre: String
- stocks: List<StockArticulo>

Sus métodos permiten consultar el nombre, obtener los registros de stock, buscar un artículo y agregar nuevos registros; asimismo, GestionComercial coordina las operaciones relacionadas con las ubicaciones y el inventario.

PagoMensualidad

Esta clase representa los pagos de mensualidad realizados por los socios; permite conservar la información de cada transacción y consultar los datos necesarios para su seguimiento.

Atributos:

- fecha: Date
- valor: double
- socio: Socio

Sus métodos permiten consultar el valor, la fecha y el socio asociado; de esta manera, los pagos se mantienen diferenciados de las ventas de productos y de las demás operaciones comerciales.

SugerenciaProducto

Representa una propuesta de producto que debe ser revisada por el personal autorizado; mantiene la información de la sugerencia y el estado del proceso de aprobación.

Atributos:

- fecha: Date
- tipo: String
- nombrePropuesto: String
- descripcion: String
- estado: String
- empleado: Empleado
- administradorRevisor: Administrador

Sus métodos permiten consultar la información y aprobar o rechazar la sugerencia; por lo tanto, la clase permite mantener un registro organizado de las propuestas comerciales y de su resolución.

Cardinalidades y relaciones

- Producto – ArticuloTienda y ProductoCafeteria: las flechas de generalización indican herencia; ambas clases especializan los datos y las operaciones comunes de Producto.
- ProductoCafeteria – Bebida y Snack: las clases concretas heredan la estructura común de los productos de cafetería.
- Usuario – Venta: un usuario puede realizar 0..* ventas; cada venta identifica al comprador.
- Venta – DetalleVenta: una venta contiene 1..* detalles; cada detalle pertenece a una venta.
- Producto – DetalleVenta: un producto puede aparecer en varios detalles (0..*); cada detalle referencia el producto vendido.
- GestionComercial – StockArticulo: el gestor administra varios registros de inventario (0..*).
- StockArticulo – UbicacionInventario: cada registro identifica la ubicación a la que pertenece; una ubicación puede contener varios registros de stock.
- Socio – PagoMensualidad: un socio puede realizar varios pagos; cada pago corresponde al socio indicado.
- SugerenciaProducto – Empleado: la sugerencia se relaciona con el empleado que la propone.
- SugerenciaProducto – Administrador: la solicitud puede relacionarse con un administrador revisor, según la multiplicidad indicada en el UML.

Con esta distribución separamos las ventas, los productos y el inventario; asimismo, facilitamos el control de existencias, el cálculo de las transacciones y el seguimiento de los pagos y las sugerencias, sin concentrar todas las operaciones en una sola clase.

8.6 Persistencia y programas de prueba

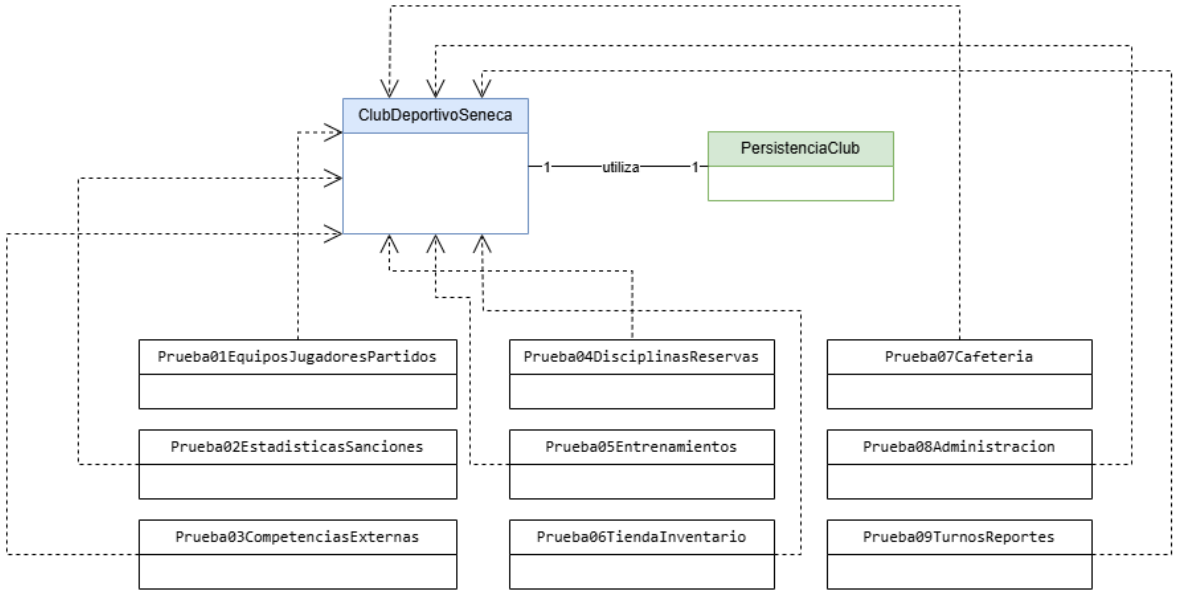


Imagen 12. UML Detallado – Persistencia y programas de pruebas

Las dos clases principales de esta vista son ClubDeportivoSeneca y PersistenciaClub:

- ClubDeportivoSeneca 1 — 1 PersistenciaClub, utiliza.

Se mantuvo la misma cardinalidad definida en la vista general, debido a que una aplicación utiliza un único componente responsable de guardar y recuperar su información.

Las nueve clases Prueba01 hasta Prueba09 se conectan mediante dependencias hacia ClubDeportivoSeneca; estas líneas no llevan cardinalidad porque los programas de prueba no forman parte del estado permanente de la aplicación, simplemente la utilizan para ejecutar escenarios.

Cada prueba corresponde a los casos definidos desde el análisis: equipos y jugadores, estadísticas y sanciones, competencias, disciplinas y reservas, entrenamientos, tienda, cafetería, administración y turnos/reportes.

9. ASIGNACIÓN DE RESPONSABILIDADES

9.1. Responsabilidades por clase

Para organizar el sistema del Club Deportivo Séneca, las responsabilidades se distribuyeron según la información que maneja cada clase y las operaciones que necesita realizar. De esta forma, cada clase tiene una función concreta y se evita concentrar toda la lógica del programa en una sola parte.

- ClubDeportivoSeneca: actúa como clase principal y coordinadora del sistema; conecta los distintos gestores y la persistencia de datos, permitiendo acceder a las funcionalidades del club desde un punto central.
- GestionUsuarios: administra los usuarios del sistema, incluyendo socios, empleados y administradores; se encarga de organizar sus datos y facilitar las operaciones relacionadas con ellos.
- GestionDeportiva: coordina las funcionalidades deportivas, como la gestión de equipos, jugadores, competiciones, disciplinas individuales, entrenamientos, partidos y estadísticas.

- GestionComercial: gestiona los productos y las operaciones comerciales; incluye las ventas, el inventario, las sugerencias de productos y los pagos de mensualidades.
- GestionTurnos: administra los turnos de los empleados y las solicitudes de cambio; también permite organizar la información necesaria para revisar estas solicitudes.
- PersistenciaClub: se ocupa de guardar y recuperar la información del sistema mediante archivos, para que los datos puedan conservarse entre distintas ejecuciones del programa.
- Usuario y sus clases derivadas: Usuario reúne los datos y comportamientos comunes de las personas que utilizan el sistema; Socio, Empleado y Administrador incorporan las características específicas de cada tipo de usuario. A su vez, Entrenador y Fisioterapeuta especializan al empleado según su función.
- Clases deportivas: clases como Equipo, Competencia, JugadorEquipo, InscripcionCompetencia y Partido representan los elementos y las relaciones de la actividad deportiva; cada una conserva los datos que le corresponden y realiza las operaciones propias de su función.
- Clases de disciplinas y entrenamiento: DisciplinaIndividual, Instalacion, Reserva, PartidoPractica, SesionEntrenamiento y Asistencia organizan la información relacionada con las actividades individuales, el uso de instalaciones, los entrenamientos y la participación de los socios.
- Clases comerciales: Producto, ArticuloTienda, ProductoCafeteria, Venta, DetalleVenta y StockArticulo representan los productos, las ventas y el inventario; permiten diferenciar los datos de cada producto de la información específica de una operación comercial.
- Clases de turnos y solicitudes: TurnoSemanal representa los turnos asignados a los empleados, mientras que SolicitudCambioTurno registra las peticiones de modificación y la información necesaria para su revisión.
- Clases de pruebas: Prueba01 a Prueba09 permiten comprobar el funcionamiento de las principales operaciones del sistema; cada prueba se centra en determinadas funcionalidades para facilitar la detección de errores.

9.2. Principios usados para distribuir responsabilidades

Para asignar las responsabilidades se tuvieron en cuenta varios principios de diseño, buscando que el sistema fuera fácil de comprender, mantener y ampliar.

Separación de responsabilidades: cada clase se ocupa de una parte concreta del sistema. Por ejemplo, GestionDeportiva coordina las operaciones deportivas, mientras que GestionComercial se encarga de las operaciones comerciales; así se evita mezclar funcionalidades que tienen objetivos diferentes.

Cohesión: se agrupan en una misma clase los datos y las operaciones que están relacionados entre sí. Por ejemplo, la clase Venta representa una operación comercial, mientras que DetalleVenta almacena la información de cada producto incluido en ella.

Bajo acoplamiento: se intenta que las clases dependan únicamente de los elementos que necesitan para cumplir su función. Los gestores coordinan operaciones entre diferentes clases cuando es necesario, sin trasladar toda la lógica del sistema a la clase principal.

Encapsulamiento: los datos de las clases se mantienen protegidos mediante atributos privados y se permite acceder a ellos o modificarlos a través de los métodos correspondientes. Esto ayuda a controlar los cambios y a mantener la coherencia de la información.

Uso de herencia: cuando varias clases comparten características, se utiliza una clase general para reunir los elementos comunes y se crean clases derivadas para representar las diferencias. Este principio se aplica, por ejemplo, en Usuario y sus especializaciones, así como en las distintas clases de productos.

Responsabilidad según la información que maneja cada clase: las comprobaciones que dependen de los datos de una entidad se sitúan preferentemente en esa clase; en cambio, las operaciones que necesitan coordinar varias entidades se gestionan desde la clase gestora correspondiente. Así, las validaciones propias de un equipo se diferencian de las comprobaciones que requieren consultar varias reservas o turnos.

Facilidad de mantenimiento y pruebas: la distribución permite revisar y comprobar las funcionalidades por separado. Si se necesita modificar una operación comercial, por ejemplo, se puede trabajar principalmente sobre las clases relacionadas con esa gestión sin tener que cambiar todo el sistema.

10. JUSTIFICACIÓN DE DECISIONES DE DISEÑO

10.1 Separación de responsabilidades

Decidimos dividir el sistema en cuatro gestores principales: usuarios, deportes, comercio y turnos, ya que cada uno se encarga de un área específica del club. De esta manera, ClubDeportivoSeneca funciona como coordinador y delega las operaciones a los gestores correspondientes, evitando que una sola clase tenga demasiadas responsabilidades.

10.2 Uso de herencia y clases abstractas

Utilizamos la herencia cuando varias clases comparten características y comportamientos, pero necesitan conservar sus propias particularidades. Por ejemplo, Usuario permite reunir los datos comunes de socios, empleados y administradores, mientras que Producto agrupa los elementos generales de la tienda y la cafetería. A su vez, las clases abstractas permiten definir una estructura común sin crear objetos de tipos que representan conceptos generales, lo cual reduce la repetición de código y facilita su mantenimiento.

10.3 Clases intermedias y relaciones

Incluimos clases intermedias cuando una relación necesita almacenar información propia, como ocurre con JugadorEquipo, InscripcionCompetencia, DetalleVenta y StockArticulo. Estas clases permiten representar con mayor precisión cómo participa un jugador en un equipo, cómo se inscribe un equipo en una competencia, qué productos componen una venta y cuántas unidades existen en cada ubicación; por esto mismo, se evita concentrar información en entidades que no son responsables de administrarla y se facilita el manejo de las cardinalidades del UML.

10.4 Distribución de las reglas de dominio

Las reglas se asignaron según la información que necesita cada clase para cumplir su función; por ejemplo, Equipo puede verificar su cupo y la disponibilidad de una camiseta, mientras que Categoria puede comprobar si un jugador cumple con el rango de edad establecido. Por otro lado, las validaciones que requieren consultar diferentes entidades se delegan a los gestores, como la disponibilidad de una instalación, la validación de una reserva o la cobertura mínima de empleados al solicitar un cambio de turno; de esta manera, las responsabilidades quedan mejor distribuidas y el funcionamiento resulta más sencillo de explicar y probar.

10.5 Organización de los diagramas e implementación

Dividimos el UML en varias vistas para mostrar las clases y sus relaciones de forma ordenada, evitando líneas sobrepuestas y diagramas demasiado grandes; aunque cada imagen representa una parte diferente, todas corresponden al mismo sistema y mantienen coherencia entre sus relaciones, cardinalidades y responsabilidades. Finalmente, decidimos utilizar estos diagramas como guía para la implementación en

Eclipse, procurando que los atributos, métodos y reglas descritos en el documento coincidan con el código Java y que las pruebas permitan comprobar el funcionamiento de las principales operaciones.

11. DISEÑO DE PERSISTENCIA

11.1 Información que debe almacenarse

El sistema necesita guardar la información que se genera durante su funcionamiento para evitar que se pierda al cerrar la aplicación. Entre estos datos se encuentran los usuarios registrados, los equipos, los jugadores, los partidos, las estadísticas, las competencias, las reservas de instalaciones, los entrenamientos y la información relacionada con la tienda, las ventas, el inventario, las mensualidades y los turnos. De esta manera, el club puede continuar trabajando con la información registrada anteriormente sin tener que introducir todos los datos nuevamente cada vez que se inicia el programa.

Para realizar este proceso, utilizamos la serialización de Java, que permite guardar objetos en archivos y luego recuperarlos. Así, podemos conservar la información de los gestores principales del sistema de una forma sencilla, sin necesidad de utilizar una base de datos externa.

11.2. Archivos utilizados

Para organizar la información, utilizamos cuatro archivos con extensión .dat, cada uno destinado a guardar una parte de los datos del club. El archivo usuarios.dat almacena la información relacionada con los usuarios registrados en el sistema, mientras que deportiva.dat contiene los datos de equipos, partidos, competencias, reservas y entrenamientos.

Por otra parte, el archivo comercial.dat guarda la información de los productos, las ventas, el inventario y las mensualidades. Finalmente, turnos.dat almacena los turnos y las solicitudes de cambio correspondientes. Esta separación permite distribuir la información según el área a la que pertenece y facilita su organización al momento de guardar o recuperar los datos.

11.3. Carga de información

Para recuperar la información almacenada, utilizamos el método cargarDatos() de la clase ClubDeportivoSeneca. Este método se encarga de acceder a los archivos de persistencia y cargar los datos correspondientes a los gestores del sistema, de manera que la aplicación pueda utilizar la información registrada en ejecuciones anteriores.

La carga de información permite que los usuarios, los datos deportivos, la información comercial y los turnos estén disponibles cuando se vuelve a ejecutar el programa; de esta forma, no es necesario registrar nuevamente toda la información cada vez que se inicia la aplicación, lo que facilita la continuidad del funcionamiento del club.

11.4. Guardado de información

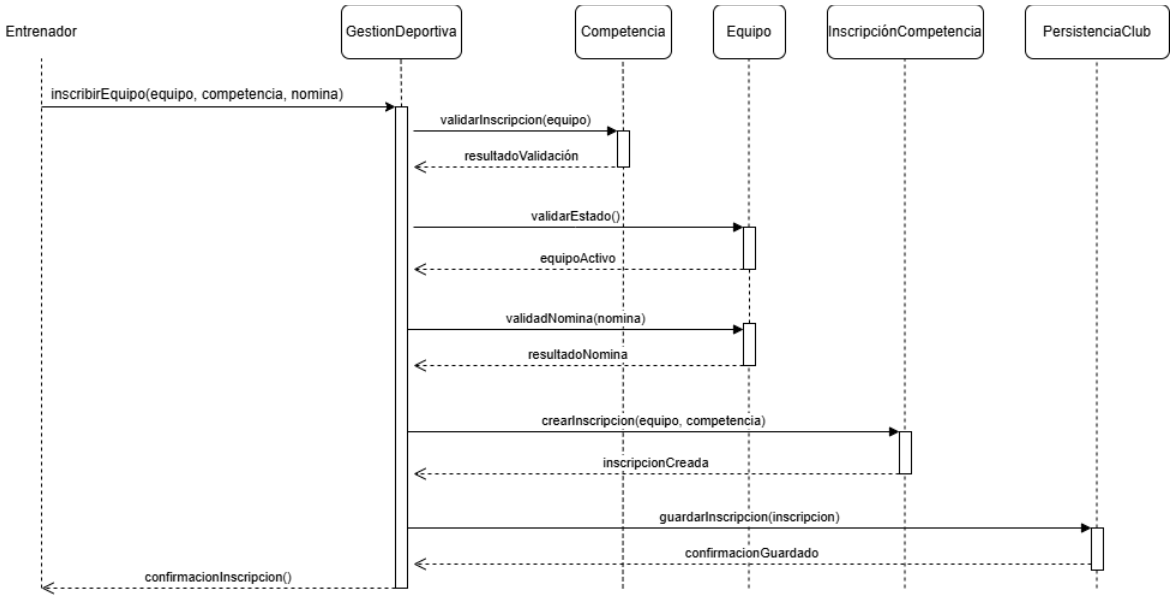
Para almacenar la información del sistema, utilizamos el método guardarDatos() de la clase ClubDeportivoSeneca. Este método trabaja junto con la clase PersistenciaClub, que se encarga de guardar los datos de los gestores principales en los archivos correspondientes mediante la serialización de Java.

El objetivo es conservar los cambios realizados durante el uso de la aplicación, incluyendo los registros y las modificaciones de la información deportiva, comercial y administrativa. Así, cuando el programa se cierre y se vuelva a iniciar, los datos podrán recuperarse mediante el método cargarDatos(), manteniendo la información organizada y evitando tener que empezar desde cero en cada ejecución.

12. DIAGRAMAS DE SECUENCIA

En esta sección se presentan los diagramas de secuencia correspondientes a los tres requerimientos críticos del sistema, con el propósito de mostrar cómo interactúan las clases para desarrollar cada proceso, qué operaciones se ejecutan y cómo se distribuyen las responsabilidades dentro del diseño; asimismo, se representan las validaciones, las respuestas y las condiciones necesarias para completar cada operación de manera adecuada.

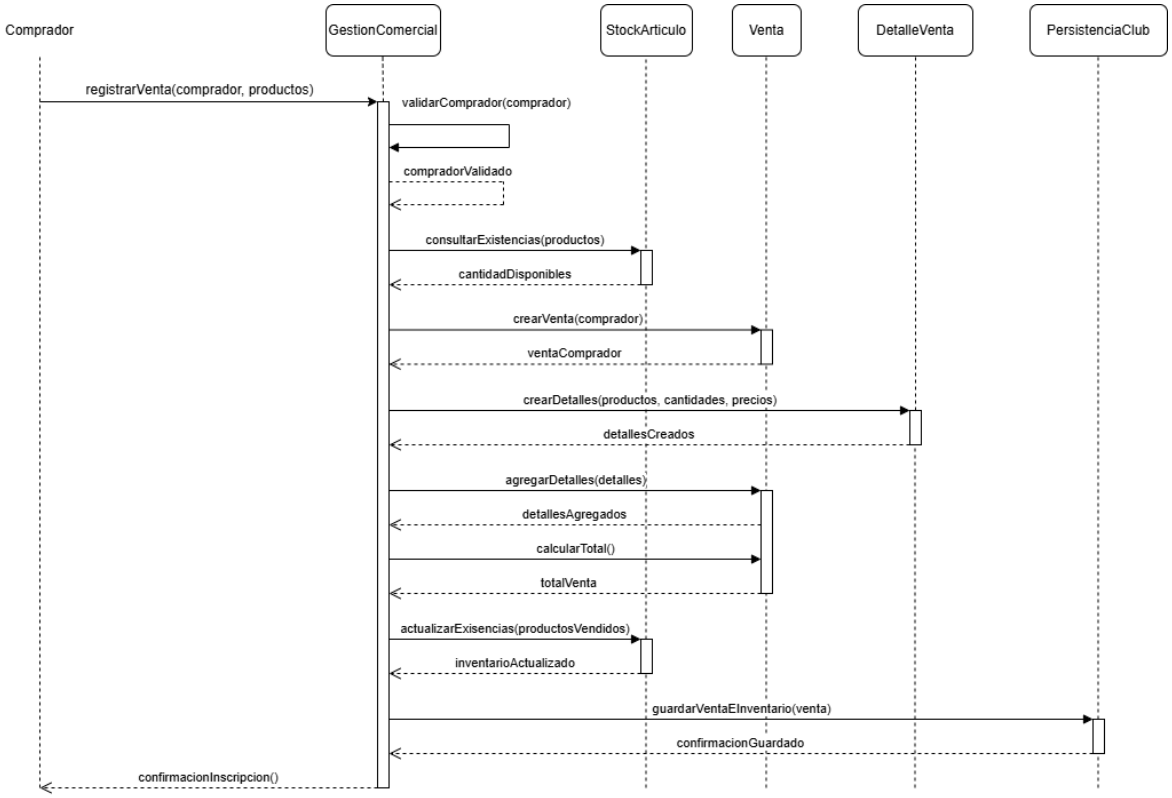
12.1 Requerimiento crítico: Inscripción de equipo en competencia



El primer diagrama representa el proceso de inscripción de un equipo en una competencia deportiva, teniendo en cuenta las validaciones necesarias antes de registrar la participación; para esto, la clase GestionDeportiva coordina las operaciones y solicita a Competencia que verifique si la inscripción es válida, mientras que Equipo comprueba su estado y la validez de la nómina de jugadores.

Adicionalmente, cuando se cumplen las condiciones requeridas, se crea una instancia de InscripcionCompetencia, se agregan los jugadores correspondientes y se guarda la información mediante PersistenciaClub; por otro lado, los fragmentos de alternativas permiten representar las situaciones en las que la competencia no está disponible, el equipo no se encuentra activo o la nómina no es válida, mientras que el ciclo muestra el registro de cada jugador.

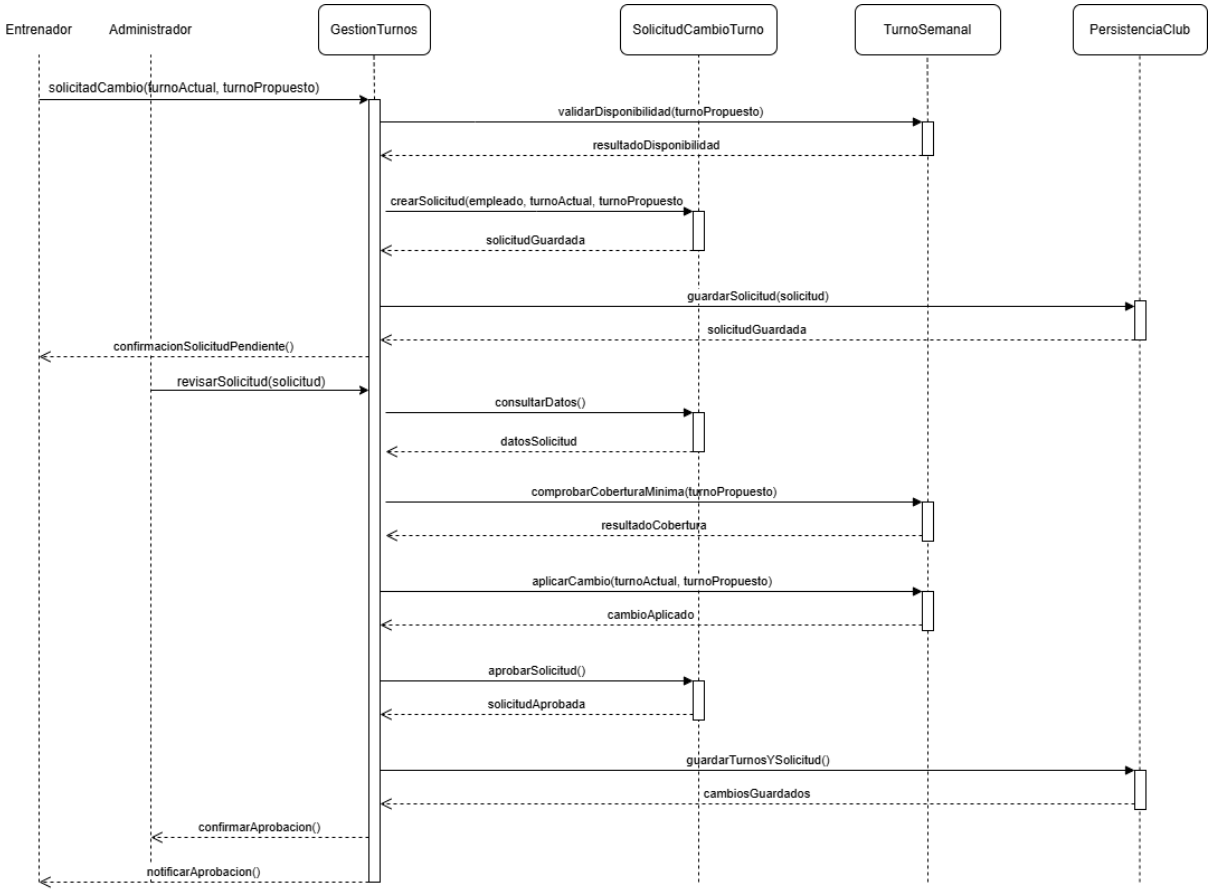
12.2 Requerimiento crítico 2: Registro de venta y actualización de inventario



El segundo diagrama describe el proceso de registro de una venta y la actualización de las existencias de los productos, mostrando cómo GestionComercial coordina las operaciones con las demás clases; inicialmente, se valida la información del comprador y se consultan las cantidades disponibles mediante StockArticulo, con el fin de comprobar que existan unidades suficientes antes de continuar.

Posteriormente, se crea la venta y se registran los detalles correspondientes a los productos, sus cantidades y sus precios; asimismo, la clase Venta calcula el valor total, mientras que StockArticulo actualiza las existencias de acuerdo con los productos vendidos. Finalmente, PersistenciaClub recibe la información para guardar la venta y los cambios realizados.

Requerimiento crítico 3: Solicitud y aprobación de cambio de turno



El tercer diagrama representa el proceso de solicitud y aprobación de un cambio de turno, en el cual GestionTurnos coordina las operaciones necesarias para comprobar si el cambio puede realizarse. En primer lugar, se consulta la disponibilidad del turno propuesto y, si existe disponibilidad, se crea la solicitud con la información del empleado y de los turnos involucrados; posteriormente, esta información se guarda mediante PersistenciaClub.

A continuación, se verifica que el cambio cumpla con la cobertura mínima requerida, condición que permite determinar si la solicitud puede aprobarse o debe rechazarse. Cuando se cumplen las condiciones, TurnoSemanal aplica el cambio y SolicitudCambioTurno registra la aprobación; en caso contrario, se actualiza el estado de la solicitud y se informa el rechazo.

13. INSTRUCCIONES DE EJECUCIÓN

Se debe desplegar src, abrir el paquete clubseneca.pruebas y seleccionar la clase de prueba que se desea ejecutar. Para iniciar una prueba, se hace clic derecho sobre la clase correspondiente y se selecciona Run As despues Java Application. Para revisar las diferentes funcionalidades, se pueden ejecutar las nueve clases de prueba disponibles, desde Prueba01EquiposJugadoresPartidos hasta Prueba09TurnosReportes; la prueba de administración también permite comprobar el guardado y la recuperación de información.

14. CONCLUSIONES

- A partir del análisis realizado en la primera entrega el cual era sobre las necesidades del Club Deportivo Séneca, pudimos reconocer cuáles eran los procesos más importantes que debía tener en cuenta el sistema; esto permitió comprender mejor cómo se organiza el club y qué aspectos se podían mejorar mediante la solución propuesta. De esta manera, se definieron los requerimientos

relacionados con la gestión deportiva y administrativa, teniendo como base las necesidades identificadas durante el análisis.

- Con la elaboración de los diagramas UML y el desarrollo del sistema en Java, logramos organizar mejor las clases y establecer las responsabilidades de cada una; así el programa tiene una estructura más clara y resulta más fácil entender cómo se relacionan sus diferentes componentes. Asimismo, este proceso permitió representar de una manera más ordenada la información del club y las reglas que se deben tener en cuenta para su funcionamiento.

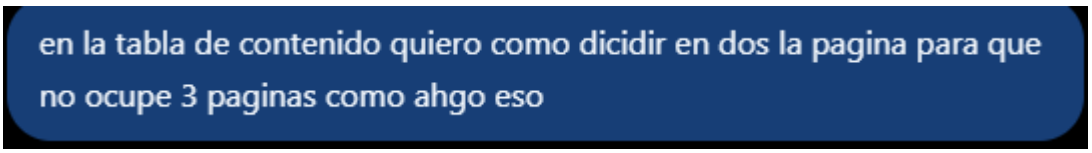
- Durante el desarrollo del sistema, fue importante revisar cómo se organizó el código y cómo se plantearon las funcionalidades, con el fin de reconocer posibles errores o aspectos que se pudieran mejorar; esto permitió tener más claridad sobre el funcionamiento del programa y sobre los ajustes que podrían ser necesarios. Además, la organización de las clases deja una base que se puede seguir mejorando en el futuro, ya que facilita realizar cambios cuando surjan nuevas necesidades del club.

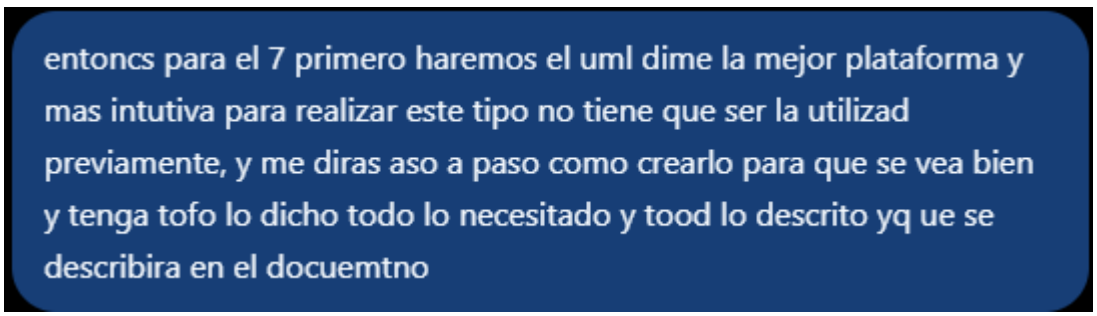
15. USO DE INTELIGENCIA ARTIFICIAL GENERATIVA

Durante el desarrollo del proyecto, se utilizó la Inteligencia Artificial Generativa como una herramienta de apoyo para organizar ideas, aclarar conceptos relacionados con el diseño UML y mejorar la redacción de algunos apartados del documento; esto permitió al grupo comprender mejor ciertos temas y presentar la información de una manera más clara y ordenada.

Sin embargo, las respuestas generadas por la herramienta se tomaron como una orientación y no como un resultado definitivo; por lo tanto, fue necesario revisar y adaptar la información de acuerdo con las necesidades del proyecto y el trabajo realizado por el grupo. De esta manera, la IA se utilizó como un complemento del proceso de aprendizaje, manteniendo la responsabilidad del grupo sobre el contenido final del documento y el desarrollo del sistema.

Prompts Usados:

- 

- 

-

Bueno antes de nada, que significa loop y alt? Osea para considerar ponerlos